Robust and Versatile Bipedal Jumping Control through Reinforcement Learning

Zhongyu Li[1], Xue Bin Peng[2], Pieter Abbeel[1], Sergey Levine[1], Glen Berseth[3,4], and Koushil Sreenath[1]
[1]University of California, Berkeley, [2]Simon Fraser University, [3]Université de Montréal, [4]Mila
Email: zhongyu_li@berkeley.edu, xbpeng@sfu.ca, pabbeel@cs.berkeley.edu, svlevine@eecs.berkeley.edu, glen.berseth@mila.quebec, koushils@berkeley.edu

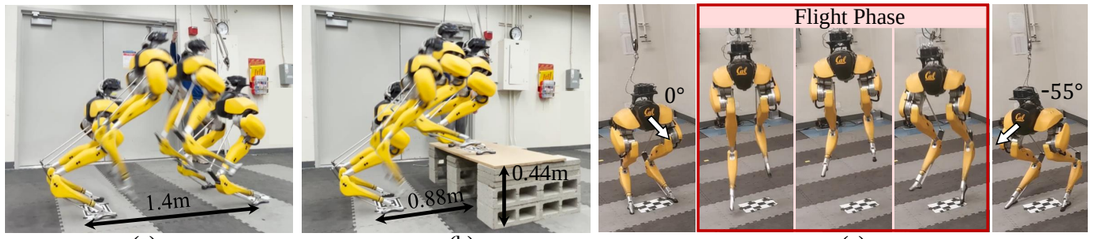


Fig. 1: Representative dynamic jumping maneuvers performed by a bipedal robot Cassie using the proposed goal-conditioned control policies. From left to right: (a) the robot jumps over 1.4 m and lands at the given target; (b) the robot jumps to a target that is 0.88 m in front of the robot and 0.44 m above the ground, and (c) the robot jumps in place while turning 55° with a command to turn 60° in place. The policies are trained in simulation and deployed on the hardware without further tuning. Video is at: https://youtu.be/aAPSZ2QFB-E.

***Abstract*—This work aims to push the limits of agility for bipedal robots by enabling a torque-controlled bipedal robot to perform robust and versatile dynamic jumps in the real world. We present a reinforcement learning framework for training a robot to accomplish a large variety of jumping tasks, such as jumping to different locations and directions. To improve performance on these challenging tasks, we develop a new policy structure that encodes the robot's long-term input/output (I/O) history while also providing direct access to a short-term I/O history. In order to train a versatile jumping policy, we utilize a multi-stage training scheme that includes different training stages for different objectives. After multi-stage training, the policy can be directly transferred to a real bipedal Cassie robot. Training on different tasks and exploring more diverse scenarios lead to highly robust policies that can exploit the diverse set of learned maneuvers to recover from perturbations or poor landings during real-world deployment. Such robustness in the proposed policy enables Cassie to succeed in completing a variety of challenging jump tasks in the real world, such as standing long jumps, jumping onto elevated platforms, and multi-axes jumps.**

I. Introduction

One question that has been lingering since the first creation of bipedal robots is: how can we enable such complex robots to traverse complex environments using agile and robust maneuvers [23, 53]? For example, creating agile controllers that enable bipedal robots to jump over a given distance or onto different elevations can enable greater mobility in unstructured environments. However, jumping is a challenging skill to control for bipedal robots. During a standing jump, the robot needs to push its body off the ground and break contact, leap into a flight phase where the robot is underactuated, and then make contact again when it lands on its legs. During landing, the robot needs to not only recover from the large impact impulse, but also stick the landing and remain standing. All of these events occur within a very short amount of time (typically less than 2 seconds). These events lead to a hybrid system that switches between modes with different contact configurations (*e.g.*, taking-off, flight, and landing). Planning and controlling such discontinuous dynamics, especially for a high-dimensional, nonlinear, and underactuated bipedal robot present a very challenging task [52]. This challenge is further compounded when the robot needs to accurately land on a given target, where the robot will have to produce the precise translational and angular momentum at take-off in order to land at the desired location [73].

Model-based optimal control (OC) frameworks have made notable progress in controlling bipedal robots, including jumping, but they depend on carefully crafted models of the robot and the complex contact dynamics [30, 39, 57]. These methods typically require manual design of task-specific control structures [9, 20, 76], and simplified dynamics models, which provides only a coarse approximation of the robot's full-order dynamics. Furthermore, pre-defined or pre-computed contact sequences are often needed to reduce online optimization complexity [46, 47, 71]. Such limitations have restricted previous model-based methods to only performing a fixed hop on torque-controlled bipedal robots like Cassie [71, 72]. By leveraging the robot's full-order dynamics, model-free reinforcement learning (RL) has shown some success in highly-dynamic locomotion control on quadrupedal robots [26, 42, 49]. However, compared to quadrupeds that are inherently

通过强化学习实现鲁棒且多功能的双足跳跃控制

Zhongyu Li[1], Xue Bin Peng[2], Pieter Abbeel[1],Sergey Levine[1],Glen Berseth[3,4],和Koushil Sreenath[1]1加州大学伯克利分校，[2]西蒙菲莎大学，[3]蒙特利尔大学´，[4]Mila 邮箱：zhongyu li@berkeley.edu, xbpeng@sfu.ca, pabbeel@cs.berkeley.edu, svlevine@eecs.berkeley.edu, glen.berseth@mila.quebec, koushils@berkeley.edu

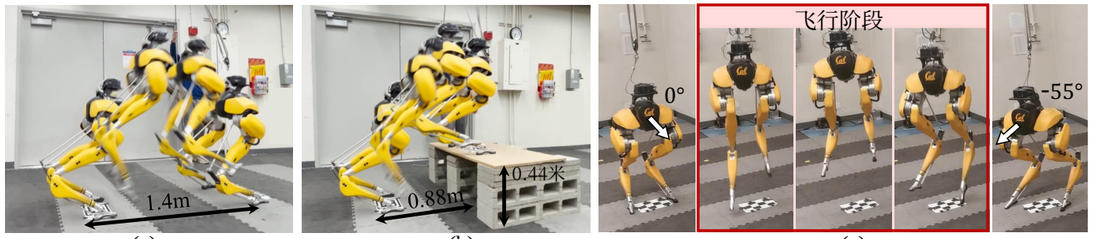


图1：双足机器人Cassie使用所提出的目标条件化控制策略执行的典型动态跳跃动作。从左至右：(a) 机器人跳过1.4米并降落在指定目标处；(b) 机器人跳跃至机器人前方0.88米、地面以上0.44米的目标位置；(c) 机器人原地跳跃并转向 55°，同时接收原地转向指令 60°。这些策略在仿真中训练并直接部署到硬件上，无需进一步调整。视频地址：https://youtu.be/aAPSZ2QFB-E。

摘要——本文旨在通过使扭矩受控的双足机器人在真实世界中执行鲁棒且多功能的动态跳跃，从而突破双足机器人的敏捷性极限。我们提出了一种强化学习框架，用于训练机器人完成多种跳跃任务，例如跳向不同位置和方向。为了提升这些具有挑战性任务的性能，我们开发了一种新的策略结构，该结构编码了机器人的长期输入/输出历史，同时提供对短期输入/输出历史的直接访问。为了训练一个多功能的跳跃策略，我们采用了一种多阶段训练方案，该方案针对不同目标包含不同的训练阶段。经过多阶段训练后，该策略可直接迁移到真实的双足Cassie机器人上。在不同任务上的训练以及探索更多样化的场景，使得策略具有高度鲁棒性，能够利用所学到的多样化机动动作，在实际部署中从扰动或不良着陆中恢复。所提策略的这种鲁棒性使Cassie能够在真实世界中成功完成各种具有挑战性的跳跃任务，例如立定跳远、跳上高台以及多轴跳跃。

I. 引言

自双足机器人诞生之初，一个挥之不去的问题便是：我们如何让这类复杂的机器人通过敏捷且稳健的动作 [23, 53]穿越复杂环境？例如，创建能够使双足机器人跳跃特定距离或到达不同高度的敏捷控制器，可以显著提升其在非结构化环境中的机动性。然而，跳跃是一项对双足机器人而言极具挑战性的控制技能。在立定跳远过程中，机器人需要将身体推离地面并脱离接触，跃入一个欠驱动的飞行阶段，然后在双腿着陆时重新建立接触。在着陆时，机器人不仅要承受巨大的冲击冲量，还必须稳定着陆并保持站立。所有这些事件都发生在极短的时间内（通常不到2秒）。这些事件导致了一个混合系统，该系统在不同接触构型的模式之间切换（例如，起飞、飞行和着陆阶段）。规划并控制这种不连续的动力学特性，尤其是对于高维度、非线性且欠驱动的双足机器人而言，是一项极具挑战性的任务 [52]。当机器人需要精确着陆在给定目标上时，这一挑战会进一步加剧，因为机器人必须在起飞时产生精确的平移和角动量，才能降落在目标位置 [73]。

基于模型的最优控制框架在控制双足机器人（包括跳跃）方面取得了显著进展，但它们依赖于精心构建的机器人模型和复杂的接触动力学 [30, 39, 57]。这些方法通常需要手动设计任务特定控制结构 [9, 20, 76], 和简化动力学模型，这只能粗略近似机器人的全阶动力学。此外，通常需要预定义或预计算的接触序列来降低在线优化复杂度 [46, 47, 71]。这些局限性使得以往基于模型的方法仅能在像Cassie这样的力矩控制双足机器人上执行固定跳跃 [71, 72]。通过利用机器人的全阶动力学，无模型强化学习在四足机器人的高度动态运动控制方面已展现出一定成功 [26, 42,49]。然而，与天生

TABLE I: Benchmark with previous work tackling jumping control using optimal control (OC) or reinforcement learning (RL) on the bipedal robot Cassie in the real world.

	Controlled Landing Pose	Apex Foot Clearence	Longest Flight Phase	Maximum Leap Distance				
				Forward	Backward	Lateral	Turning	Elevation
Aperiodic Hop by OC [71]	No	0.18m	0.42s	∼0.5m*	0m	0m	0°	0m
Aperiodic Hop by OC [72]	No	0.15m	0.33s*	∼0.3m*	0m	0m	0°	0m
Periodic Hop by RL [60]	No	∼0.16m*	0.33s*	∼0.5m*	0m	0m	0°	∼0.15m*
Ours (Aperiodic Jump by RL)	**Yes**	**0.47m**	**0.58s**	**1.4m**	**-0.3m**	**±0.3m**	**±55°**	**0.44m**

*Not provided in the paper and the listed value is roughly estimated based on the comparison with the background environment in the accompanying video.

more stable, RL-based methods still struggle when applied to bipedal robots for more dynamic and aggressive maneuvers in the real world. Given that the most relevant prior RL-based system is only able to perform periodic hopping on Cassie [60], it remains an open question how more dynamic bipedal skills can be achieved in the real world, such as standing long jumps that could be more challenging than periodic motions [20, Sec. I].

A. Objective of this Paper

In this paper, our objective is to explore the possibility of creating a robust and versatile jumping controller for a bipedal robot that enables it to land at different target locations, as shown in Fig. 1, and validating the advantages brought by learning with different maneuvers using reinforcement learning. We refer a *skill* to a kind of legged locomotion method, such as walking, jumping, or running. We further denote a *task* as using a locomotion skill to accomplish a *goal*. For instance, one task could be the task of walking (skill) at a desired speed (goal), or as in this work, achieving the task of jumping (skill) to a target configuration (goal). We use the term *goal-conditioned* to refer to a policy that can perform a variety of jumping tasks, such as jumping over various desired distances and/or directions, conditioned on the given goal. We hypothesize that, by exploring different jumping tasks, a versatile jumping controller can be more robust as it allows the robot to leverage more diverse learned tasks to maintain stability during dynamic maneuvers. For example, in order to recover from a large ground impulse on landing, the robot can quickly switch to another learned task, such as a hop to a different location, which can allow for a more graceful recovery than simply continuing with the original behavior.

B. Contributions

The core contribution of this work is the the development of the first system that enables a life-sized torque-controlled bipedal robot to perform *versatile jumping maneuvers with controlled landing locations* in the real world. The robot is first trained in simulation with reinforcement learning and domain randomization. Training does not require an explicit contact sequence, and the learning algorithm automatically develops different contact sequences for different jumping goals. In order to successfully transfer the learned skill for such dynamic maneuvers from simulation to the real world, we utilize two new design decisions. First, we present a new policy structure that encodes the robot's long-term Input-Output (I/O) history while also providing direct access to a short-term I/O history. By training the model in an end-to-end manner, we show that such a structure can outperform previously proposed architectures (Fig. 5). Second, we demonstrate that training the controller for a diverse range of goals improves the robustness of the controller by using the maneuvers learned from different jumping tasks to recover from unstable states. We show that this robustness cannot be easily obtained by training only for a single jumping goal (Fig. 6). Combining these two techniques, we enable the bipedal Cassie robot to perform (1) several aggressive jumps, such as a long jump (1.4m ahead) and a high jump onto a 0.44m-tall platform (Fig. 1), (2) various agile multi-axes bipedal jumps (Fig. 7, Fig. 9), and (3) to utilize diverse learned jumping maneuvers to recover from external perturbations or impacts (Fig. 9), in the real world. We hope this paper can serve as a step forward for enabling more diverse, dynamic, and robust legged locomotion skills.

II. Related Work

Prior work tackling dynamic locomotion skills such as jumping with legged robots can be broadly categorized as corresponding to model-based optimal control (OC) and model-free reinforcement learning (RL). Table I compares our work with the most related prior efforts on the bipedal robot Cassie.

1) Model-based optimal control for legged jumping: Prior model-based methods for legged jumping control usually build up a layered optimization scheme, which includes offline trajectory optimization with detailed models of the robot's dynamics and ground contacts [10, 12, 46, 63], and online controllers that leverage simplified models of the robot's dynamics [45, 55, 65, 72]. In order to optimize trajectories for jumping, which needs to switch among modes with different underlying dynamics, there are two commonly employed solutions: (i) relying on human-specified contact sequences [9, 19, 29, 47, 71], which is not scalable to different jump distances and/or directions, or (ii) leveraging contact-implicit optimization [8, 14, 33, 52, 77] which plans through contacts to avoid breaking the trajectory or using computationally expensive mixed-integer programming [1, 11, 12]. However, due to the computational challenges of optimization, both of the above-mentioned methods are still limited to offline computation for legged robots. As we show in this work, by training with different jumping goals offline, Cassie can automatically generate the appropriate contact sequences during online execution for achieving robust jumping.

In the case of controllers for aperiodic dynamic jumps, many previous efforts required a separate landing controller to stabilize the robot from the large landing impacts [20, 25, 46, 71]. However, this approach usually requires a contact estimator and needs to fuse the noisy robot proprioceptive measurements

表I：与先前在真实世界中针对双足机器人Cassie使用最优控制（OC）或强化学习（RL）进行跳跃控制的研究基准测试对比。

	受控着陆姿态	Apex 足部离地高度	最长飞行阶段	最大跳跃距离				
				向前	向后	横向	转向	抬升
OC [71]的非周期跳跃	No	0.18米	0.42秒	∼0.5米*	0m	0m	0°	0m
OC [72]的非周期跳跃	No	0.15米	0.33秒*	∼0.3米*	0m	0m	0°	0m
基于强化学习的周期性跳跃 [60]	No	∼0.16米*	0.33秒*	∼0.5米*	0m	0m	0°	∼0.15米*
我们的（基于强化学习的非周期性跳跃）	**Yes**	**0.47米**	**0.58秒**	**1.4m**	**-0.3米**	**±0.3米**	**±55°**	**0.44米**

*论文中未提供该数值，所列数值是基于与附带视频中背景环境的对比粗略估算得出。

更稳定的四足机器人相比，基于强化学习的方法在应用于双足机器人以在真实世界中执行更具动态性和侵略性的动作时仍然面临困难。鉴于最相关的先前基于强化学习的系统仅能在Cassie上执行周期性跳跃 [60], , 如何在真实世界中实现更具动态性的双足技能（例如可能比周期性运动更具挑战性的立定跳远）仍然是一个悬而未决的问题 [20, 。第I节]。

A. 本文目标

本文旨在探索为双足机器人构建一个鲁棒且多功能的跳跃控制器的可能性，使其能够着陆于不同的目标位置（如图1所示），并通过强化学习验证不同机动动作学习所带来的优势。我们将技能定义为一种腿部运动方式，例如行走、跳跃或奔跑。我们进一步将任务定义为运用某种运动技能来实现一个目标。例如，一个任务可以是以期望速度行走（技能）的任务（目标），或者如本文所述，实现跳跃（技能）至目标构型（目标）的任务。我们使用术语目标条件化来指代一种能够根据给定目标执行多种跳跃任务（例如跳过各种期望距离和/或方向）的策略。我们假设，通过探索不同的跳跃任务，一个多功能的跳跃控制器可以更加鲁棒，因为它允许机器人利用更多样化的已学任务来在动态机动过程中保持稳定性。例如，为了从着陆时巨大的地面冲击中恢复，机器人可以快速切换到另一个已学任务，例如跳跃到不同位置，这比简单地继续执行原始行为能够实现更平稳的恢复。

B. 贡献

本文的核心贡献在于开发了首个系统，使真实世界中具备全尺寸力矩控制的双足机器人能够执行具有受控着陆位置的多功能跳跃动作。该机器人首先在仿真环境中通过强化学习和域随机化进行训练。训练过程无需显式指定接触序列，学习算法会自动针对不同的跳跃目标发展出相应的接触序列。为了成功将此类动态动作的学习技能从仿真迁移至真实世界，我们采用了两个新的设计决策。首先，我们提出了一种新的策略结构，该结构在编码机器人长期输入输出历史的同时，也提供对短期输入输出历史的直接访问。通过以端到端方式训练模型，我们

证明了此类结构能够超越先前提出的架构（图5）。其次，我们证明了针对多样化目标范围训练控制器，通过利用从不同跳跃任务中学到的动作从不稳定状态中恢复，能够提升控制器的鲁棒性。我们展示了这种鲁棒性无法通过仅针对单一跳跃目标进行训练轻易获得（图6）。结合这两种技术，我们使双足Cassie机器人在真实世界中能够执行：(1) 多次激进跳跃，例如跳远（向前1.4米）和跳高至0.44米高的平台（图1）；(2) 各种敏捷的多轴双足跳跃（图7、图9）；(3) 利用多样化的习得跳跃动作从外部扰动或冲击中恢复（图9）。我们希望本文能成为推动实现更多样化、更动态、更鲁棒的腿部运动技能向前迈出的一步。

II. 相关工作

先前针对跳跃等动态运动技能的研究，可大致分为基于模型的最优控制和基于模型的强化学习两类。表I将我们的工作与双足机器人Cassie上最相关的先前研究进行了比较。

1) 基于模型的最优控制在腿式跳跃中的应用：先前基于模型的腿式跳跃控制方法通常构建一个分层优化方案，该方案包括利用机器人动力学和地面接触的详细模型进行离线轨迹优化 [10, 12, 46, 63], , 以及利用机器人动力学简化模型的在线控制器 [45, 55, 65, 72]。为了优化需要在不同动力学模式间切换的跳跃轨迹，通常有两种常用解决方案：(i) 依赖人工指定的接触序列 [9, 19, 29, 47, 71], , 这种方法难以扩展到不同的跳跃距离和/或方向；或(ii) 利用接触隐式优化 [8, 14, 33, 52, 77] , 通过接触进行规划，以避免轨迹中断或使用计算成本高昂的混合整数规划 [1, 11, 12]。然而，由于优化带来的计算挑战，上述两种方法在腿式机器人上仍局限于离线计算。正如我们在本工作中所示，通过针对不同跳跃目标进行离线训练，Cassie能够在在线执行过程中自动生成合适的接触序列，以实现稳健的跳跃。

在非周期动态跳跃的控制方法中，许多先前的工作需要单独的着陆控制器来稳定机器人，使其免受巨大着陆冲击的影响 [20, 25, 46, 71]。然而，这种方法通常需要接触估计器，并需要融合嘈杂的机器人本体感觉测量数据

to estimate the contact states, which on its own could be a challenging problem [22, 38, 51]. Furthermore, while there are a few prior attempts addressing *precise* jumping control on low-dimensional single-legged robots [66, 73] and a quadrupedal robot [45], mostly in simulation [45, 66], most prior work on bipedal jumping only focus on single vertical jumps with the landing location not controlled [20, 65, 71, 72]. In this work, the proposed jumping controller demonstrates the capacity to control the landing pose of the bipedal robot without position feedback or explicit contact estimation.

2) Model-free RL for legged locomotion control: In recent years, we have seen exciting progress in using deep RL to learn locomotion controllers for quadrupedal robots [5, 16, 34, 40] and bipedal robots [7, 37, 54, 59, 70, 75] in the real world. Since it is challenging in general to learn a single policy with RL to perform various tasks [28], many prior works focus on learning a single-goal policy [6, 42, 49, 74] for legged robots, such as just forward walking at a constant speed [16, 31, 69]. There have been efforts to obtain more versatile policies, such as walking at different velocities using different gaits, while following different commands [17, 18, 35, 54], which requires more extensive tuning due to the lack of a gait prior. Providing the robot with different reference motions for different goals can be helpful, but requires additional parameterization of the reference motions (*e.g.*, a gait library) [3, 24, 27, 37], policy distillation [70], or a motion prior [15, 50, 67]. There is also a line of research to explicitly provide contact sequences for legged robots [4, 41, 58, 60]. However, such methods are prescriptive and provide little opportunity for the robot to deviate from the contact plan, limiting the flexibility with which it can respond to perturbations. In this work, we show that a versatile policy can enhance the robustness of a jumping policy by intelligently employing a variety of learned tasks to react to perturbations.

3) Sim-to-real transfer for legged robots: To tackle sim-to-real transfer for RL-based methods, some works have sought to directly train policies directly in the real world [21, 62, 68], but most of the prior work, especially for dynamic skills, leverages a simulator to train the legged robot with extensive dynamics randomization [48] and then zero-shot transfer to the real world [16, 31, 34, 37, 60] or finetune with real-world data [27, 49, 61]. Since performing rollout on the hardware of human-scale bipedal robots is expensive, we use the zero-shot transfer method. In order to realize this, there are two widely-adopted techniques: (i) end-to-end training a policy by providing the robot with a proprioceptive short-term history [16, 24, 37] or long-term history [48, 58, 59], (ii) teacher-student training that first obtains a teacher policy with privileged information of the environment by RL, then uses this policy to supervise the training of a student policy that only has access of onboard-available observations [18, 26, 31, 34, 40, 75], which shows advantages over the end-to-end training method [31, 32]. However, here we show that, for the dynamic control of bipedal robots, by training the robot in an end-to-end way with a newly-proposed policy structure, we can realize a better learning performance over the teacher-student method which separates the training process and results in increased training time and data.

III. Background and Preliminaries

In this section, we provide a brief introduction to our experimental platform, Cassie and the background of goal-conditioned reinforcement learning.

A. Floating-base Model of Cassie

We use Cassie as the experimental platform in this work. Cassie (see Fig. 1) is a life-sized bipedal robot and is around 1.1 meter tall, with a weight of 31 Kg. It is a dynamic and underactuated system, with 5 actuated motors (abduction q_1, rotation q_2, thigh q_3, knee q_4, and toe q_7) and 2 passive joints (shin q_5 and tarsus q_6) connected by leaf springs on its Left and Right leg. We denote the motor positions as $\mathbf{q}_m = [q^{L/R}_{1,2,3,4,7}] \in \mathbb{R}^{10}$. The 6 degree of freedom (DoF) floating base (pelvis) can be represented with translational positions (sagittal q_x, lateral q_y, vertical q_z) and rotational positions (roll q_ψ, pitch q_θ, and yaw q_ϕ). In total, the robot has 20 DoFs $\mathbf{q} \in \mathbb{R}^{20}$. For more details about Cassie's configuration, we refer readers to [71, Fig. 2]. The observable joint positions on Cassie are denoted as $\mathbf{q}^o = [q_{\psi,\theta,\phi}, \mathbf{q}_m, \dot{q}_{x,y,z}, \dot{\mathbf{q}}_m] \in \mathbb{R}^{26}$, which can be obtained from onboard joint encoders and IMUs, while the base linear velocity $\dot{q}_{x,y,z}$ can be estimated with an EKF [70].

B. RL Background and Goal-Conditioned Policy

We formulate the locomotion control problem as a Markov decision process (MDP). At each timestep t, the agent (*i.e.*, the robot) observes the environment state $\mathbf{s}_t$, and the policy π produces a distribution over the actions, $\pi(\mathbf{a}_t|\mathbf{s}_t)$, conditioned on the state. The agent then executes the action $\mathbf{a}_t$ sampled from the policy, interacts with the environment, makes an observation of the environment's new states $\mathbf{s}_{t+1}$, and receives a reward r_t. The objective of RL is to maximize the expected accumulative reward (return) the agent received over the course of an episode $\mathbb{E}[\Sigma_{t=0}^{T}\gamma^t r_t]$ where γ is a discount factor and T is the episode length. In order to obtain a policy that can accomplish different goals, we provide a goal $\mathbf{c}$ which parameterizes the task and the policy $\pi(\mathbf{a}_t|\mathbf{s}_t, \mathbf{c})$ is then also conditioned on the given goal $\mathbf{c}$ to perform different tasks.

Task Parameterization: In our jumping task, the goal $\mathbf{c}$ specifies target commands for a desired jump $\mathbf{c} = [c_x, c_y, c_z, c_\phi]$, which consists of the target location $c_{x,y}$ on the horizontal plane, elevation c_z in the vertical direction, and turning direction c_ϕ after the agent lands, calculated based on the robot's pose before the jump, *i.e.*, in the local frame of robot's starting pose. Please note that the change in elevation c_z is defined as the change of the floor height, instead of the change of the robot's base height.

IV. Multi-Stage Training for Versatile Jumps

We now describe our multi-stage training framework for developing goal-conditioned jumping policies. The training environment is developed in a simulation of Cassie using MuJoCo [13, 64].

来估计接触状态，这本身可能就是一个具有挑战性的问题 [22, 38, 51]。此外，尽管有一些先前的尝试针对低维度的单腿机器人 [66, 73]和四足机器人 [45]，（主要在仿真中 [45, 66]，）实现了精确的跳跃控制，但大多数关于双足跳跃的先前工作仅关注着陆位置不受控制的单次垂直跳跃 [20, 65, 71, 72]。在这项工作中，所提出的跳跃控制器展示了无需位置反馈或显式接触估计即可控制双足机器人着陆姿态的能力。

2) 用于腿式运动控制的无模型强化学习：近年来，我们见证了利用深度强化学习在真实世界中为四足机器人 [5, 16, 34, 40]和双足机器人 [7, 37, 54, 59, 70, 75]学习运动控制器的激动人心的进展。由于通常很难通过强化学习训练一个单一策略来执行各种任务 [28]，，许多先前的工作专注于为腿式机器人学习单目标策略 [6, 42, 49, 74] ，例如仅以恒定速度向前行走 [16, 31, 69]。已有一些努力旨在获得更多多功能策略，例如使用不同步态以不同速度行走，同时遵循不同指令 [17, 18, 35, 54]，，但由于缺乏步态先验，这需要更广泛的调参。为机器人提供针对不同目标的不同参考运动可能有所帮助，但需要对参考运动进行额外的参数化（例如，步态库） [3, 24, 27, 37], 、策略蒸馏 [70], 或运动先验 [15, 50, 67]。还有一系列研究致力于为腿式机器人明确提供接触序列 [4, 41, 58, 60]。然而，此类方法具有规定性，几乎不给机器人偏离接触计划的机会，从而限制了其应对扰动的灵活性。在这项工作中，我们展示了一种多功能策略能够通过智能地利用各种已学任务来应对扰动，从而增强跳跃策略的鲁棒性。

3) 腿式机器人的仿真到现实迁移：为了解决基于强化学习方法的仿真到现实迁移问题，一些工作尝试直接在真实世界中训练策略 [21, 62, 68]，，但大多数先前的工作，特别是针对动态技能，利用仿真器通过广泛的动力学随机化来训练腿式机器人 [48] ，然后零样本迁移到真实世界 [16, 31, 34, 37, 60] 或使用真实世界数据进行微调 [27, 49, 61]。由于在人形尺度双足机器人的硬件上进行 rollout 成本高昂，我们采用了零样本迁移方法。为了实现这一点，有两种广泛采用的技术：(i) 通过向机器人提供本体感受的短期历史 [16, 24, 37] 或长期历史 [48, 58, 59],进行端到端训练策略；(ii) 师生训练，首先通过强化学习利用环境的特权信息获得教师策略，然后使用该策略监督仅能访问机载可用观测的学生策略的训练 [18,26, 31, 34, 40, 75], ，这显示出优于端到端训练方法的优势 [31, 32]。然而，我们在此表明，对于双足机器人的动态控制，通过使用新提出的策略结构以端到端方式训练机器人，我们可以实现比

师生方法更好的学习性能，该方法分离了训练过程，导致训练时间和数据增加。

III. 背景与预备知识

在本节中，我们将简要介绍实验平台 Cassie 以及目标条件强化学习的背景知识。

A. Cassie 的浮动基座模型

本研究使用 Cassie 作为实验平台。Cassie（见图1）是一款真人大小的双足机器人，高约1.1米，重31公斤。它是一个动态欠驱动系统，每条腿上有5个驱动电机（外展 q_1、旋转 q_2、大腿 q_3、膝盖 q_4和脚趾 q_7）以及2个由板簧连接的被动关节（小腿 q_5 和跗骨 q_6）。我们将电机位置记为$\mathbf{q}_m = [q^{L/R}_{1,2,3,4,7}] \in$。浮 $\mathbb{R}^{10}$ 6 动基座（骨盆）的自由度可以用平移位置（矢状 q_x、横向q_y、垂直 q_z）和旋转位置（横滚 q_ψ、俯仰 q_θ、偏航 q_ϕ）来表示。机器人总共有20个自由度$\mathbf{q} \in \mathbb{R}^{20}$。关于 Cassie 配置的更多细节，请读者参阅 [71,图 2]。Cassie 上可观测的关节位置记为$\mathbf{q}^o = [q_{\psi,\theta,\phi}, \mathbf{q}_m, \dot{q}_{x,y,z}, \dot{\mathbf{q}}_m] \in \mathbb{R}^{26}$，这些数据可通过机载关节编码器和惯性测量单元获取，而基座线速度$\dot{q}_{x,y,z}$ 则可通过扩展卡尔曼滤波器 [70]进行估计。

B. 强化学习背景与目标条件策略

我们将运动控制问题建模为马尔可夫决策过程。在每个时间步 t，智能体（即，机器人）观察环境状态 $\mathbf{s}_t$，策略 π 基于状态生成动作上的分布 $\pi(\mathbf{a}_t|\mathbf{s}_t)$。随后，智能体执行从策略中采样的动作 $\mathbf{a}_t$ ，与环境交互，观测环境的新状态 $\mathbf{s}_{t+1}$，并接收奖励 r_t。强化学习的目标是最大化智能体在一个回合 $\mathbb{E}[\Sigma_{t=0}^{T}\gamma^t r_t]$ 内获得的期望累积奖励（回报），其中 γ 是折扣因子， T 是回合长度。为了获得能够完成不同目标的策略，我们提供一个目标 $\mathbf{c}$ 来参数化任务，策略 $\pi(\mathbf{a}_t|\mathbf{s}_t,\mathbf{c})$ 随后也基于给定目标 $\mathbf{c}$ 进行条件化，以执行不同的任务。

任务参数化：在我们的跳跃任务中，目标 $\mathbf{c}$ 指定了期望跳跃的目标指令 $\mathbf{c} = [c_x, c_y, c_z, c_\phi]$，该指令包含水平面上的目标位置 $c_{x,y}$ 、垂直方向上的抬升 c_z ，以及智能体落地后的转向方向 c_ϕ ，这些均基于机器人跳跃前的姿态计算，即，在机器人起始姿态的局部坐标系中。请注意，抬升 c_z 的变化定义为地面高度的变化，而非机器人基座高度的变化。

IV. 多功能跳跃的多阶段训练

我们现在描述用于开发目标条件跳跃策略的多阶段训练框架。训练环境是在使用MuJoCo [13, 64]的Cassie仿真中开发的。

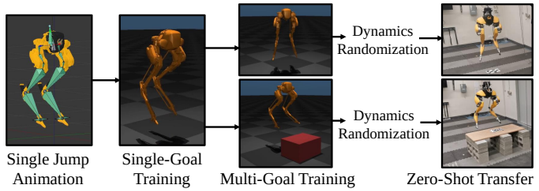


Fig. 2: The schematic to train the robot to perform versatile jumping skills in the real world starting with a reference motion of a single jumping animation. This framework consists of three stages. In the first stage, we focus on training the robot to imitate the animation while performing a single jump from scratch. After the robot is good at achieving the single goal, we randomize the goal (to land at different locations and different turning directions/elevations) that is assigned to the robot during each training episode. After these two stages of training, we extensively randomize the dynamics properties of the environment in simulation in order to improve the robustness of the robot during the zero-shot transfer from sim to real.

A. Overview of the Multi-Stage Training Schematic

Our goal is to develop a locomotion control policy for jumping skills that can perform targeted jumps to different locations. However, due to the challenging nature of jumping, it can be difficult to directly train a policy to perform a large variety of jumps from scratch. We observed that training policies from scratch to perform a large variety of jumps tends to lead to the robot adopting very conservative behaviors or even failing to learn to jump. Therefore, we use a multi-stage training scheme that consists of 3 stages, as illustrated in Fig. 2: (1) single-goal training, (2) multi-goal training, and (3) dynamics randomization. All stages of training are performed in simulation, but as we show in our experiments, the resulting models can then be directly deployed on a real Cassie robot. In *Stage 1*, the model is trained on a single goal $\mathbf{c}$, *i.e.*, jumping in place. This stage of training results in a policy that is trained from scratch to specialize in a single task in simulation. Next, in *Stage 2*, the goal $\mathbf{c}$ is randomized every episode to train the robot to jump to different targets. In this stage, the focus is primarily on performing the commanded task in the simulated environment. Finally, in *Stage 3*, we introduce extensive domain randomization on the simulation environment, while also randomizing the goal, in order to improve the robustness and generalization of the policy for sim-to-real transfer. At each stage, the reward and episode designs of the MDP may vary in order to produce more effective policies for the objectives of the given stage.

In the rest of this section, we focus on explaining the details of Stages 2&3 in training, which share the same reward and episode design and cover both multi-goal training and domain randomization. Stage 1 training, which is different in the choice of hyperparameters, is detailed in Appendix A.

B. Reference Motion

To initialize the training process, we provide a *single* jumping reference motion. The reference motion is a human-authored animation of Cassie jumping in place, created in a $3D$ creation suite [36], as presented in Fig. 2. This animated jump has an apex foot height of 0.5 m and an apex pelvis height of 1.1 m and has a timespan T_J of 1.66 second (*i.e.*, $T_J = 1.66$). This reference motion is only a kinematically-feasible motion for the agent and is not optimized to be dynamically feasible. After the end of the jumping animation, the reference motion will be set to a fixed standing pose for the robot.

TABLE II: A list of components of reward r_t which is a weighted summation of the listed items. The weight of each term is scheduled based on the jumping phase and training stage.

Reward Component r	**Weight w**			
	Stage 1		**Stage 2, 3**	
	$t \leq T_J$	$t > T_J$	$t \leq T_J$	$t > T_J$
Reference Motion Tracking				
Motion position: $r(\mathbf{q}_m, \mathbf{q}_m^r(t))$	15	15	7.5	15
Pelvis height: $r(q_z, q_z^r(t) + c_z)$	5	5	3	3
Foot height: $r(e_z, e_z^r(t) + c_z)$	10	10	10	10
Task Completion				
Pelvis position: $r(q_{x,y}, c_{x,y})$	12.5	12.5	15	15
Pelvis velocity: $r(\dot{q}_{x,y}, \dot{q}_{x,y}^d)$	0	3	12.5	12.5
Orientation: $r(q_{\psi,\theta,\phi}, [0, 0, c_\phi])$	12.5	12.5	10	10
Angular rate: $r(q_{\psi,\theta,\phi}, [0, 0, \dot{q}_\phi^d])$	3	3	10	10
Smoothing				
Ground Impact: $r(F_z, 0)$	5	0	10	0
Torque Consumption: $r(\boldsymbol{\tau}, 0)$	3	3	3	15
Motor velocity: $r(\dot{\mathbf{q}}_m, 0)$	0	15	0	25
Joint acceleration: $r(\ddot{\mathbf{q}}, 0)$	3	10	0	5
Change of action: $r(\mathbf{a}_t, \mathbf{a}_{t+1})$	0	0	10	10

C. Reward

The design of the reward function is important to encourage the robot to jump with agility. We further spilt the reward within a stage into two phases: before landing ($t \leq T_J$) and after ($t > T_J$), and the reward needs to vary based on these phases because the desired robot's behavior is different: performing aggressive jump versus stationary standing.

We here define a function:

$$r(u, v) = \exp(-\alpha||u - v||_2^2) \tag{1}$$

where $r(u, v) \in (0, 1]$ defines a reward component that encourage the two vector u and v to be as close as possible, scaled by $\alpha > 0$ that balance the units. The reward r_t the agent receives at each timstep is a weighted summation of different components, $r_t = (\mathbf{w}/||\mathbf{w}||_1)^T\mathbf{r} \in [0, 1]$. The component vector $\mathbf{r}$ and weight vector $\mathbf{w}$ are detailed in Table II. The reward used here consists of three main components: reference motion tracking, task completion, and smoothing term.

The agent is encouraged to track the reference motor position by $r(\mathbf{q}_m, \mathbf{q}_m^r(t))$, pelvis height by $r(q_z, q_z^r(t) + c_z)$, and foot height $r(e_z, e_z^r(t)+c_z)$ at current timestep t. However, as recorded in Table II, the reference motion tracking term has a relatively small weight during the multi-goal training because we want the agent to infer diverse maneuvers, such as jumping to different locations, and the jumping-in-place reference motion may be no longer reasonable.

The task completion reward, on the contrary, is designed to dominate others during multi-goal training. We first include the $r(q_{x,y}, c_{x,y})$ and $r(q_{\psi,\theta,\phi}, [0, 0, c_\phi])$ to encourage the agent to reach the desired location and orientation and stay there

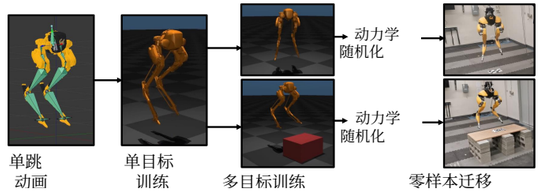


图2：训练机器人在真实世界中执行多样化跳跃技能的示意图，从单个跳跃动画的参考运动开始。该框架包含三个阶段。在第一阶段，我们专注于训练机器人从零开始模仿动画并执行单跳。当机器人能够熟练完成单目标后，我们在每个训练回合中随机化分配给机器人的目标（使其降落在不同位置，并实现不同的转向方向/抬升）。经过这两个阶段的训练后，我们广泛随机化仿真环境中的动力学属性，以提升机器人在从仿真到真实的零样本迁移过程中的鲁棒性。

A. 多阶段训练示意图概述

我们的目标是开发一种用于跳跃技能的运动控制策略，使其能够执行针对不同位置的目标跳跃。然而，由于跳跃本身具有挑战性，直接训练一个策略从零开始执行多种跳跃可能非常困难。我们观察到，从零开始训练策略以执行多种跳跃往往会导致机器人采取非常保守的行为，甚至无法学会跳跃。因此，我们采用了一种包含3个阶段的多阶段训练方案，如图2所示：(1) 单目标训练，(2) 多目标训练，以及 (3) 动力学随机化。所有阶段的训练均在仿真中进行，但正如我们在实验中所展示的，最终得到的模型可以直接部署在真实的Cassie机器人上。在阶段*1*中，模型针对单个目标$\mathbf{c}$进行训练，即原地跳跃。此阶段的训练结果是一个从零开始训练、专门用于仿真中单一任务的策略。接下来，在阶段*2*中，每个回合的目标$\mathbf{c}$ 被随机化，以训练机器人跳跃到不同的目标点。在此阶段，重点主要是在仿真环境中执行指定的任务。最后，在阶段*3*中，我们在仿真环境中引入广泛的域随机化，同时随机化目标，以提高策略的鲁棒性和泛化能力，从而实现仿真到现实迁移。在每个阶段，马尔可夫决策过程的奖励和回合设计可能会有所不同，以便为给定阶段的目标生成更有效的策略。

在本节剩余部分，我们将重点解释训练中阶段2和阶段3的细节，这两个阶段共享相同的奖励和回合设计，并涵盖多目标训练和域随机化。阶段1训练在超参数选择上有所不同，详见附录A。

B. 参考运动

为了初始化训练过程，我们提供了一个单一跳跃参考运动。该参考运动是一个由人类创作的

表II：奖励组件列表 r_t，该奖励是所列各项的加权求和。每个项的权重根据跳跃阶段和训练阶段进行调度。

奖励组件 r	**权重 w**			
	阶段1		**阶段2, 3**	
	$t \leq T_J$	$t > T_J$	$t \leq T_J$	$t > T_J$
参考运动跟踪				
运动位置： $r(\mathbf{q}_m, \mathbf{q}_m^r(t))$	15	15	7.5	15
骨盆高度： $r(q_z, q_z^r(t) + c_z)$	5	5	3	3
脚部高度： $r(e_z, e_z^r(t) + c_z)$	10	10	10	10
任务完成				
骨盆位置： $r(q_{x,y}, c_{x,y})$	12.5	12.5	15	15
骨盆速度： $r(\dot{q}_{x,y}, \dot{q}_{x,y}^d)$	0	3	12.5	12.5
朝向： $r(q_{\psi,\theta,\phi}, [0, 0, c_\phi])$	12.5	12.5	10	10
角速率： $r(q_{\psi,\theta,\phi}, [0, 0, \dot{q}_\phi^d])$	3	3	10	10
平滑				
地面冲击： $r(F_z, 0)$	5	0	10	0
扭矩消耗： $r(\boldsymbol{\tau}, 0)$	3	3	3	15
电机速度： $r(\dot{\mathbf{q}}_m, 0)$	0	15	0	25
关节加速度： $r(\ddot{\mathbf{q}}, 0)$	3	10	0	5
动作变化： $r(\mathbf{a}_t, \mathbf{a}_{t+1})$	0	0	10	10

Cassie原地跳跃的动画，在 $3D$创作套件 [36], 中制作，如图2所示。该动画跳跃的最高点足部高度为0.5米，最高点骨盆高度为1.1米，时间跨度为 T_J 1.66秒（即， $T_J = 1.66$）。该参考运动仅对智能体而言是运动学可行的运动，并未优化为动力学可行。在跳跃动画结束后，参考运动将设置为机器人的固定站立姿态。

C. 奖励

奖励函数的设计对于鼓励机器人敏捷跳跃至关重要。我们进一步将阶段内的奖励分为两个阶段：着陆前（ $t \leq T_J$）和着陆后（$t > T_J$），并且奖励需要根据这些阶段而变化，因为期望的机器人行为不同：执行激进跳跃与静止站立。

我们在此定义一个函数：

$$r(u, v) = \exp(-\alpha||u - v||_2^2) \tag{1}$$

其中 $r(u, v) \in (0, 1]$ 定义了一个奖励组件，用于鼓励两个向量 u 和 v 尽可能接近，并由 $\alpha > 0$ 进行缩放以平衡单位。智能体在每个时间步获得的奖励 r_t 是不同组件的加权求和，即 $r_t = (\mathbf{w}/||\mathbf{w}||_1)^T\mathbf{r} \in [0, 1]$。分量向量$\mathbf{r}$ 和权重向量$\mathbf{w}$ 详见表II。此处使用的奖励由三个主要组件构成：参考运动跟踪、任务完成和平滑项。

智能体被鼓励在当前时间步 t跟踪参考电机位置（通过 $r(\mathbf{q}_m, \mathbf{q}_m^r(t))$）、骨盆高度（通过 $r(q_z, q_z^r(t) + c_z)$）以及脚部高度（通过 $r(e_z, e_z^r(t)+c_z)$）。然而，如表II所示，在多目标训练期间，参考运动跟踪项的权重相对较小，因为我们希望智能体推断出多样化的动作，例如跳跃到不同位置，而原地跳跃参考运动可能不再合理。

相反，任务完成奖励旨在多目标训练中主导其他奖励。我们首先引入 $r(q_{x,y}, c_{x,y})$和 $r(q_{\psi,\theta,\phi}, [0, 0, c_\phi])$，以鼓励智能体到达目标位置和朝向并保持在那里

after it lands in order to accomplish the assigned task $\mathbf{c}$. Furthermore, pelvis linear velocity tracking $r(\dot{q}_{x,y}, \dot{q}^d_{x,y})$ and angular rate tracking $r(\dot{q}_{\psi,\theta,\phi}, [0,0,\dot{q}^d_\phi])$ are introduced to shape the sparse task reward, where $\dot{q}^d_{x,y} = c_{x,y}/T_J$ and $\dot{q}^d_\phi = c_\phi/T_J$. Moreover, although the task does not include the pelvis roll and pitch angle $q_{\psi,\theta}$, minimizing them to zero can help to stabilize the robot's pelvis.

We further introduce a smoothing term that is less important than task completion but with a larger weight than the motion tracking term. For example, we encourage the robot to produce less ground impact force F_z during its jump by $r(F_z, 0)$, to damp the body's oscillation after it lands by motor velocity reward $r(\dot{\mathbf{q}}_m, 0)$ and joint acceleration reward $r(\ddot{\mathbf{q}}, 0)$, and to be more energy efficient by $r(\boldsymbol{\tau}, 0)$. Moreover, the importance of having a stationary standing pose is highlighted by having a relatively large weight on torque consumption and motor velocity reward after the robot lands ($t > T_J$). This is because the introduction of dynamics randomization in Stage 3 will make the environment noisy and cause oscillation in body pose during standing. We also introduce an additional component in Stages 2&3, the change of action reward $r(\mathbf{a}_t, \mathbf{a}_{t+1})$, to further smooth the aggressive maneuver the robot may conduct to jump over a long distance.

Remark 1: Although the switching time (T_J) of the reward is fixed and not tuned for jumping to different locations, as we show in our experiments, the robot learns different flight times for different targets (Fig. 7b).

D. Episode Design

Having a careful design of the reward may not be enough since it is challenging to encourage the agent to jump. The robot may keep failing to stabilize itself while learning to jump. Therefore, the robot may easily adopt very conservative but stable behaviors because it can quickly improve the return in this way. For example, the robot may just stand or just jump in place without completing the task, and can still have some suboptimal return. To prevent this, we note that a cautious design of the episode can also facilitate the training of dynamic jumping maneuvers.

In the stages for multi-goal training, the maximum episode length is set to 2500 timestep which lasts 76 seconds. During such an episode, the robot is commanded to jump to a random target after a random time interval of standing, and these random values are uniformly sampled. The task is sampled from $c_x \sim U(-0.5, 1.5)$ m, $c_y \sim U(-1.0, 1.0)$ m, $c_z \sim U(-0.5, 0.5)$ m, and $c_\phi \sim U(-100°, 100°)$, and standing phase distribution is $U(1, 15)$ second. Such a "jump $\leftrightarrow$ stand" switch is repeated. Such a design can improve the robustness of the learned policy to different initial states by performing repeat jumps over an episode. Moreover, compared to Stage 1 where the agent is asked to jump at $t = 0$, starting from Stage 2, there is a high probability the robot will start with a standing skill at each episode.

The episode will be terminated earlier if the robot falls over (pelvis height $q_z < 0.55$ m or the tarsus joints hit the ground) to prevent it from having further rewards. We also emphasize the importance of foot height tracking and task completion to the robot by terminating the episode earlier if: (i) the foot height tracking error $|e_z - e^d_z|$ is larger than the bound E_e which is set at 0.32 m, or (ii) the robot does not arrive at the given target after it lands ($t > T_J$) when $[||q_{x,y} - c_{x,y}||_2, |q_\phi - c_\phi|] > E_t$ where $E_t = [0.35\text{ m}, 35°]$. Please note that we have a relatively small task completion error bound E_t while we have a large tolerance on the foot height tracking error E_e. Using such a design, the robot is allowed to deviate from the reference foot trajectory to find a better foot height trajectory for different tasks. The robot will also have more incentive to complete the task by landing close to the target and having more rewards in a longer episode.

Remark 2: The larger choice of the foot tracking error tolerance E_e also allows the robot to perform small hops after it lands. The robot is encouraged to stand due to the foot height tracking reward but can dynamically switch to hop, including hopping to different places as long as staying within E_t, for better robustness. We do not specifically train or encourage the agent to deviate from the assigned task for robustness.

In the first stage of training, a single jump introduces an inductive bias into the policy towards performing jumping behaviors. In later stages of training, by combining the early termination conditions and motion imitation reward, this inductive bias leads the robot to favor jumping to different targets, instead of using other skills such as walking.

E. Dynamics Randomization

In order to succeed during the sim-to-real transfer, we introduce extensive randomization on dynamics parameters of the environment in Stage 3. The dynamics properties that are randomized are listed completely in Table III in Appendix B. During training at this stage, at each episode, the value of each dynamics parameter is uniformly sampled from the range listed in Table III. We consider three sources that cause the sim-to-real gap: (1) modeling errors, (2) sensor noise, and (3) communication delay between the high-level computer running the RL policy and the robot's low-level computer.

In order to robustify the policy to the modeling errors, we randomize the floor friction, robot's joint damping, link mass and inertia, and the position of the link's Center of Mass (CoM). Specifically, to deal with the error of motor dynamics between the simulation and hardware, we have a larger upper bound of the joint damping (4 times the default value) to approximate the motor aging issues on the hardware. We also randomize the PD gains used in the joint-level PD controllers (since our policy outputs target motor positions). The range is $\pm 30\%$ of the default value. Such a change is able to diversify the motor responses the robot is trained on and enhance the robustness to the change of motor dynamics during hardware deployment. Furthermore, specific to Cassie whose leg has leaf springs to connect the passive joints $q_{5,6}$, the parameters of the springs are important because they will have significant displacement during the taking-off and landing phases. Therefore, we introduce a 20% uncertainty on

在落地后完成指定任务$\mathbf{c}$。此外，引入骨盆线速度跟踪 $r(\dot{q}_{x,y}, \dot{q}^d_{x,y})$和角速率跟踪 $r(\dot{q}_{\psi,\theta,\phi}, [0,0,\dot{q}^d_\phi])$来塑造稀疏任务奖励，其中$\dot{q}^d_{x,y} = c_{x,y}/T_J$和$\dot{q}^d_\phi = c_\phi/T_J$。此外，尽管任务不包含骨盆翻滚角和俯仰角$q_{\psi,\theta}$，但将它们最小化至零有助于稳定机器人的骨盆。

我们进一步引入了一个平滑项，其重要性低于任务完成，但权重高于运动跟踪项。例如，我们鼓励机器人在跳跃过程中通过 $r(F_z,0)$减少地面冲击力 F_z ，通过电机速度奖励 $r(\dot{\mathbf{q}}_m, 0)$和关节加速度奖励 $r(\ddot{\mathbf{q}}, 0)$抑制落地后身体的振荡，并通过 $r(\boldsymbol{\tau}, 0)$提高能效。此外，通过在机器人落地后对扭矩消耗和电机速度奖励赋予较大权重（$t > T_J$），强调了保持静止站立姿态的重要性。这是因为在阶段3中引入动力学随机化会使环境变得嘈杂，并导致站立时身体姿态出现振荡。我们还在阶段2和阶段3中引入了一个额外组件——动作变化奖励 $r(\mathbf{a}_t, \mathbf{a}_{t+1})$，以进一步平滑机器人为了跳跃长距离而可能采取的激进动作。

备注*1*：尽管奖励的切换时间（T_J）是固定的，且未针对跳跃到不同位置进行调优，但正如我们的实验所示，机器人学会了针对不同目标的不同飞行时间（图7b）。

D. 回合设计

精心设计奖励可能还不够，因为鼓励智能体跳跃本身就具有挑战性。机器人在学习跳跃的过程中，可能始终无法稳定自身。因此，机器人很容易采取非常保守但稳定的行为，因为这样能快速提升回报。例如，机器人可能只是站立或原地跳跃，而没有完成任务，但仍能获得一些次优的回报。为防止这种情况，我们注意到，谨慎设计回合也有助于训练动态跳跃动作。

在多目标训练的阶段中，最大回合长度设置为2500个时间步，持续76秒。在这样的一个回合中，机器人在随机站立时间间隔后被指令跳跃到一个随机目标，这些随机值均为均匀采样。任务从 $c_x \sim U(-0.5,1.5)$米、$c_y \sim U(-1.0,1.0)$米、$c_z \sim U(-0.5,0.5)$米和 $c_\phi \sim U(-100°,100°)$中采样，站立阶段分布为 $U(1, 15)$秒。这种"跳跃$\leftrightarrow$ 站立"的切换会重复进行。这种设计可以通过在回合中执行重复跳跃，提高学习策略对不同初始状态的鲁棒性。此外，与阶段1中要求智能体在 $t = 0$ 时刻跳跃不同，从阶段2开始，机器人在每个回合开始时大概率会以站立技能启动。

如果机器人摔倒（骨盆高度低于 $q_z < 0.55$ 米或跗关节触地），该回合将提前终止，以防止其获得更多奖励。我们还强调

足部高度跟踪和任务完成对机器人的重要性，通过以下条件提前终止回合：（i）足部高度跟踪误差 $|e_z - e^d_z|$ 大于设定的界限 E_e（即0.32米），或（ii）机器人在着陆后（$t > T_J$）未到达给定目标，当 $[||q_{x,y} - c_{x,y}||_2, |q_\phi - c_\phi|] > E_t$ 且 $E_t = [0.35$ 米 $35°]$时。请注意，我们的任务完成误差界限 E_t 相对较小，而对足部高度跟踪误差 E_e 的容忍度较大。通过这种设计，机器人被允许偏离参考足部轨迹，以寻找更适合不同任务的足部高度轨迹。同时，机器人将有更多动力通过靠近目标着陆并在更长的回合中获得更多奖励来完成任务。

备注*2*：选择较大的足部跟踪误差容限 E_e 也允许机器人在落地后进行小幅跳跃。由于足部高度跟踪奖励，机器人被鼓励站立，但可以动态切换到跳跃，包括在 E_t范围内跳跃到不同位置，以获得更好的鲁棒性。我们并未专门训练或鼓励智能体为了鲁棒性而偏离指定任务。

在训练的第一阶段，单跳为策略引入了执行跳跃行为的归纳偏置。在后续训练阶段，通过结合提前终止条件和运动模仿奖励，这种归纳偏置引导机器人倾向于跳跃到不同目标，而不是使用行走等其他技能。

E. 动力学随机化

为了在仿真到现实迁移中取得成功，我们在阶段3对环境动力学参数引入了广泛的随机化。被随机化的动力学属性完整列于附录B的表III中。在此阶段训练期间，每个回合中，每个动力学参数的值均从表III所列范围内均匀采样。我们考虑了导致仿真到现实差距的三个来源：(1) 建模误差，(2) 传感器噪声，以及(3) 运行强化学习策略的高层计算机与机器人底层计算机之间的通信延迟。

为了增强策略对建模误差的鲁棒性，我们对地面摩擦、机器人的关节阻尼、连杆质量和惯性以及连杆质心位置进行随机化处理。具体而言，为了处理仿真与硬件之间电机动力学的误差，我们设置了更大的关节阻尼上限（默认值的4倍），以近似模拟硬件上的电机老化问题。同时，我们还对关节级PD控制器中使用的PD增益进行随机化（因为我们的策略输出目标电机位置）。随机化范围为默认值的 $\pm 30\%$。这种变化能够使机器人训练时接触到的电机响应多样化，并增强硬件部署过程中对电机动力学变化的鲁棒性。此外，针对腿部通过板簧连接被动关节的Cassie机器人$q_{5,\ 6}$，弹簧参数至关重要，因为在起飞和着陆阶段它们会产生显著位移。因此，我们引入了20%的不确定性

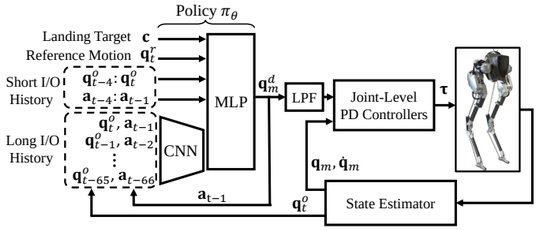


Fig. 3: The architecture of the goal-conditioned jumping policy π_θ. The policy outputs the desired motor positions $\mathbf{q}_m^d$, which are used by joint-level PD controllers to generate the motor torques $\boldsymbol{\tau}$ on the robot. The input to the policy includes the goal $\mathbf{c}$, which specifies the landing targets, the reference motion $\mathbf{q}_t^r$, which provides the robot a short preview of the reference trajectory, and a short 4-timestep history of the robot's input (robot's action $\mathbf{a}_{t-1}$) and output (robot's feedback $\mathbf{q}_t^o$). The policy is also provided with a long-term 2-second I/O history, which is first encoded by a 1D CNN. The policy updates at 33 Hz while the rest runs at 2 kHz.

spring stiffness during training. We empirically found that the randomization of the motor dynamics and spring stiffness has a non-trivial effect to succeed during the sim-to-real for the bipedal jumping skills.

The sensor noise from joint encoders, IMU, and estimation error of the base linear velocity are simulated as a Gaussian noise whose mean is sampled in Table III at each episode.

V. Training Setup

We now build up our control policy by optimizing reinforcement learning through the multi-stage training pipeline.

A. Policy Architecture

Our policy π_θ is represented by a deep neural network with parameters θ. As shown in Fig. 3, it has two components, a base network represented by a multilayer perceptron (MLP), and a long-term history encoder represented by a 1D convolutional neural network (CNN). The policy operates at 33 Hz. Each action $\mathbf{a}_t$ specifies the target motor positions $\mathbf{q}_m^d$ for the robot. The action is first passed through a Low Pass Filter (LPF) [15, 37, 49], which smooths the motor targets before being applied to joint-level PD controllers and further complements the smoothing rewards. The PD controllers operates at 2 kHz, to generate motor torques $\boldsymbol{\tau} \in \mathbb{R}^{10}$ for driving the movements of the joints.

The input to the policy at timestep t contains four components: the goal $\mathbf{c}$ introduced in Sec. III-B, a preview of the reference trajectory $\mathbf{q}_t^r$, a short-term history of previous actions and states (robot's Input/Output), and a long-term I/O history of the last 2-second. The preview of the reference trajectory $\mathbf{q}_t^r = [q_z^r(t), \mathbf{q}_m^r(t+1), \mathbf{q}_m^r(t+4), \mathbf{q}_m^r(t+7)]$ provided in the robot's observation contains the current reference pelvis height $q_z^r(t)$ and reference motor positions $\mathbf{q}_m^r$ sampled at 1, 4, and 7 future timesteps. Providing a segment of the future reference trajectory as input provides the policy with more information such as future joint position, velocity and other higher order terms, which has been used in [37, 49, 61]. To close the control loop, we provide the robot direct access to a short-term I/O history of the robot $(\mathbf{q}_{t-4:t}^o, \mathbf{a}_{t-4:t-1})$ in the previous 4 timesteps (about 0.12 second). The I/O history enables the policy to infer the dynamics of the system from past observations. The task $\mathbf{c}$, reference motion $\mathbf{q}_t^r$, and the short-term I/O history at current timestep t are directly passed as inputs to the base MLP.

For sim-to-real transfer, a short-term history may not be enough to provide adequate information to control dynamic maneuvers on a high-dimensional system. For example, during a jump, the landing event is affected by the angular momentum gained before the take-off, and the interval between these two events can be much longer than the 0.12 second. Therefore, we include an additional input in the form of a long-term I/O history of the past 2 seconds, which contains 66 timesteps of past observations and actions measurements $(\mathbf{q}_{t-65:t}^o, \mathbf{a}_{t-66:t-1})$. The timespan of this long I/O history is designed to cover the duration of a jump to help the policy implicitly infer the robot's dynamics, traveled trajectory, and contacts. To encode this long sequence of observations, we use a 1D CNN to compress it into a latent representation before providing it as an input to the base MLP. As we will see in Fig. 5, *both* the long-term and short-term history are needed for better learning performance.

In this work, the CNN encoder consists of 2 hidden layers whose [kernel size, filter size, stride size] are $[6, 32, 3]$ and $[4, 16, 2]$ with *relu* activation and no padding, respectively. The result of the CNN is flattened and concatenated into the inputs of the base MLP. The MLP has two hidden layers with 512 *tanh* units, followed by an output layer representing the mean of a Gaussian action distribution with a fixed standard deviation of $0.1I$.

B. Training Details

Empirically, we found that simultaneously performing both turning and jumping to different elevations is very difficult for Cassie, which does not have a torso. Due to this hardware limitation, we choose to train two separate goal-conditioned policies: a *flat-ground policy* that is specialized for jumping without elevation changes, *i.e.*, $c_z = 0$, and a *discrete-terrain policy* that is trained to jump onto platforms with different elevations without turning ($c_\phi = 0$).

Proximal Policy Optimization (PPO) [56] is used to train all policies π_θ in simulation, with a value function represented by a 2-layered MLP, which has an access to the ground truth observations. Due to the differences in the complexity of the different training stages, the three stages are trained with 6k, 12k, and 20k iterations, respectively. Each iteration collects a batch of 65536 samples.

VI. Simulation Validation

Having introduced our methodology for training goal-conditioned jumping policies, we will next validate the proposed method in simulation (MuJoCo). In this section, we aim to address two questions: (1) what are the advantages of the proposed policy architecture compared to models used in prior work, (2) whether training with multiple tasks can further improve the robustness of the policy over single-goal training,

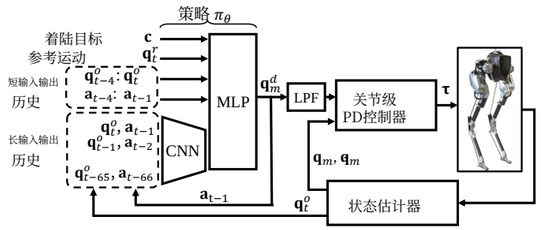


图3：目标条件跳跃策略 π_θ的架构。策略输出期望的电机位置$\mathbf{q}_m^d$，由关节级PD控制器生成机器人上的电机扭矩 $\boldsymbol{\tau}$。策略的输入包括目标$\mathbf{c}$（指定着陆目标）、参考运动$\mathbf{q}_t^r$（为机器人提供参考轨迹的短时预览），以及机器人输入（机器人动作$\mathbf{a}_{t-1}$）和输出（机器人反馈$\mathbf{q}_t^o$）的4时间步历史。策略还接收一个长期2秒的输入输出历史，该历史首先由一维卷积神经网络编码。策略以33 Hz更新，其余部分以2 kHz运行。

对训练过程中的弹簧刚度进行随机化。根据经验，我们发现电机动力学和弹簧刚度的随机化对于双足跳跃技能在仿真到现实迁移中的成功具有不可忽视的影响。

来自关节编码器、惯性测量单元的传感器噪声以及基座线速度的估计误差被模拟为高斯噪声，其均值在每个回合中从表III采样。

五、训练设置

现在，我们通过多阶段训练流程优化强化学习，从而构建控制策略。

A. 策略架构

我们的策略 π_θ 由一个参数为 θ的深度神经网络表示。如图3所示，它包含两个组件：一个由多层感知机（MLP）表示的基础网络，以及一个由一维卷积神经网络（CNN）表示的长期历史编码器。该策略以33 Hz的频率运行。每个动作$\mathbf{a}_t$ 指定了机器人的目标电机位置$\mathbf{q}_m^d$。该动作首先通过一个低通滤波器（LPF） [15, 37, 49]，，在应用于关节级PD控制器之前对电机目标进行平滑处理，并进一步补充平滑奖励。PD控制器以2 kHz的频率运行，生成电机扭矩 $\boldsymbol{\tau} \in \mathbb{R}^{10}$ 以驱动关节运动。

在时间步 t ，策略的输入包含四个组成部分：目标 $\mathbf{c}$ （在第三节B部分介绍）、参考轨迹的预览$\mathbf{q}_t^r$、先前动作和状态（机器人的输入/输出）的短期历史，以及过去2秒的长期输入输出历史。参考轨迹的预览$\mathbf{q}_t^r = [q_z^r(t), \mathbf{q}_m^r(t+1), \mathbf{q}_m^r(t+4), \mathbf{q}_m^r(t+7)]$ 包含在机器人的观测中，提供了当前参考骨盆高度 $q_z^r(t)$以及在1、4和7个未来时间步采样的参考电机位置$\mathbf{q}_m^r$ 。将未来参考轨迹的一段作为输入提供给策略，使其获得更多信息，例如未来的关节位置、速度和其他高阶项，这一方法已在 [37, 49, 61]中使用。为了闭合控制回路，我们让机器人直接访问

机器人过去4个时间步（约0.12秒）的短期输入输出历史（$\mathbf{q}_{t-4:t}^o$,$\mathbf{a}_{t-4:t-1}$）。输入输出历史使策略能够从过去的观测中推断系统的动态特性。任务$\mathbf{c}$、参考运动$\mathbf{q}_t^r$以及当前时间步 t 的短期输入输出历史直接作为输入传递给基础多层感知机。

对于仿真到现实迁移，短期历史可能不足以提供足够的信息来控制高维系统上的动态机动。例如，在跳跃过程中，着陆事件受到起飞前获得的角动量的影响，而这两个事件之间的间隔可能远长于0.12秒。因此，我们以过去2秒的长期输入输出历史的形式包含一个额外输入，其中包含66个时间步的过去观测和动作测量（$\mathbf{q}_{t-65:t}^o$,$\mathbf{a}_{t-66:t-1}$）。该长期输入输出历史的时间跨度旨在覆盖一次跳跃的持续时间，以帮助策略隐式推断机器人的动力学、运动轨迹和接触。为了编码这一长序列的观测，我们使用一维卷积神经网络将其压缩为潜在表示，然后将其作为输入提供给基础多层感知机。正如我们将在图5中看到的，长期和短期历史对于更好的学习性能都是必需的。

在本工作中，卷积神经网络编码器包含2个隐藏层，其 [卷积核大小、滤波器大小、步长] 分别为 $[6, 32, 3]$ 和$[4, 16, 2]$ ，采用*relu* 激活函数且无填充。卷积神经网络的结果被展平并拼接至基础多层感知机的输入中。该多层感知机具有两个隐藏层，每层包含512个*tanh* 单元，其后是一个输出层，表示具有固定标准差$0.1I$的高斯动作分布的均值。

B. 训练细节

根据经验，我们发现对于没有躯干的Cassie而言，同时执行转向和跳跃至不同高度非常困难。由于这一硬件限制，我们选择训练两个独立的目标条件策略：一个专门用于无高度变化跳跃的平地策略，即， $c_z = 0$；以及一个训练用于在不转向的情况下跳跃至不同高度平台的离散地形策略 （$c_\phi = 0$）。

近端策略优化（PPO） [56] 用于在仿真中训练所有策略 π_θ，其价值函数由一个2层多层感知机表示，该感知机可以访问真实观测。由于不同训练阶段的复杂度差异，三个阶段分别以6000次、12000次和20000次迭代进行训练。每次迭代收集65536个样本的批次。

VI. 仿真验证

在介绍了我们用于训练目标条件跳跃策略的方法后，接下来将在仿真环境（MuJoCo）中验证所提方法。本节旨在探讨两个问题：（1）与先前工作中使用的模型相比，所提策略架构具有哪些优势；（2）与单目标训练相比，多任务训练是否能通过让机器人利用更多样化的动作从不稳定状态或未知扰动中恢复，从而进一步提升策略的鲁棒性。

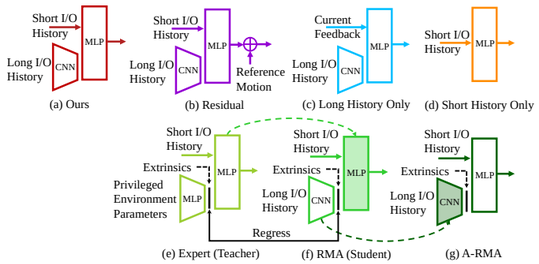


Fig. 4: Illustration of the baseline policy structures used to train the policy for bipedal jumping. (a) Ours: proposed structure as discussed in detail in Fig. 3. (b) Residual policy that has the same input structure as our method but outputs a residual term adding to the reference motor position [34, 70]. (c) Long History Only policy that only has the access to a long-term I/O history (we still provide robot immediate feedback to the base, as suggested by Peng et al. [48]). (d) Short History Only policy that is only provided with a short-term I/O history [37]. We also compare with the RMA [31]/Teacher-Student [34] training strategy where an (e) expert policy with access to privileged environment information (Table III) is first trained by RL and is later utilized to train (f) RMA (student) policy by supervised learning. The RMA can be further finetuned using (g) A-RMA [32] by RL. While the short I/O history is not included in the original RMA [31] or TS [34], it is included in this benchmark to have a fair comparison. The blocks are shaded if their parameters are not updated. The dash lines indicate that parameters are copied.

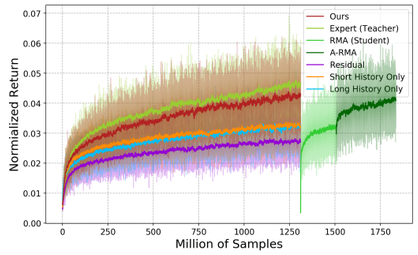


Fig. 5: Benchmark of learning curves trained by different policy structures in Stage 3 (multi-goal training with dynamics randomization). The curves are the average normalized returns trained with 3 random seeds while the colored areas enclose the min and max values obtained among different seeds. The normalized return is calculated by the return divided by the max episode length and in the range of $[0, 1]$. Our method shows similar performance as the expert policy which is used to supervise RMAs and has access to the privileged environment parameters. The A-RMA shows the second-best performance but it requires significantly more samples compared to the proposed methods, followed by RMA. The policies with short history only or long history only show a similar learning performance but are a bit worse than RMA in terms of returns. The residual policy shows the worst performance because the reference motion added to the policy's action prevents the agent from exploring more diverse maneuvers.

A. Baselines

To answer the first question, we benchmark our proposed policy architecture with several baselines illustrated in Fig. 4. All policies are trained with multiple goals, *i.e.*, jumping to different landing locations and turning directions with no change of elevation, using the training schematic shown in Fig. 2, and are trained with 3 different random seeds. The details of baseline models are described in Appendix C.

To address the second question, we obtained two single-goal policies using the proposed policy structure, as detailed below:

- **Single Goal**: a policy that is trained on a single jumping-in-place task and extensive dynamics randomization as listed in Table III.
- **Single Goal w/ Perturbation**: a policy similar to the single-goal policy but is also trained with a randomized perturbation wrench (6 DoF) applied on the robot pelvis. The external forces and torques are sampled uniformly from $[-20\text{N}, -5\text{Nm}]$ to $[20\text{N}, 5\text{Nm}]$ and are applied on the robot's pelvis for a random time interval ranging from $[0.1, 2.0]$ second.

We compare these baselines based on two metrics: (1) learning performance in Sec. VI-B and (2) the ability to generalize to dynamics parameters that lie outside of the training distributions in Sec. VI-C. These two metrics are important for the sim-to-real transfer because the first one shows how well the policy can perform during training and the second evaluates robustness to changes in the environment, which are not considered during training, as can be the case during sim-to-real transfer.

B. Policy Structure Choice

The learning curves from Stage 3 (multi-goal learning with dynamics randomization) using our policy structure and baselines are presented in Fig. 5. The learning curves at early training stages (learning a single task in Stage 1 and multiple tasks in Stage 2) are available in Fig. 10 in Appendix D. The same hyperparameters and reward functions are used for every training stage.

According to Fig. 5 (and Fig. 10), the residual structure drawn as the purple curve shows the worst learning performance over all the training stages. The reason is the reference motion we provided is a dynamically-infeasible animation, which may cause the robot to spend more effort learning to correct these default motions, and prevents it from exploring more diverse trajectories and inferring the motion that is outside of the range of the reference motion.

The baselines using short history only (orange curve) and long history only (blue curve) show a similar learning performance. But if we combine these two by providing the policy with a long history encoder *and* direct access to short history, which results in our method, the learning performance can be enhanced to a large extent, as drawn as the red curves in Fig. 5. This showcases that, providing the policy with a long history is not enough because the robot may need immediate feedback which may be hidden from the long-history encoder. Providing the policy with direct access to the short history can address it and the agent can learn to utilize both information.

Remark 3: We note that there is other work using RNNs with LSTM [48, 58, 59] or TCN [34] to encode the long-term I/O history. We hypothesize that providing the policy by allowing the robot to utilize more diverse maneuvers to recover from unstable states or unknown perturbations.

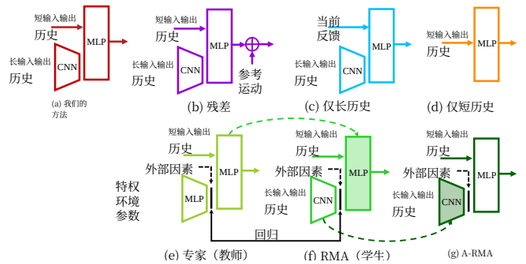


图4：用于训练双足跳跃策略的基线策略结构示意图。(a) 我们的方法：如图3详细讨论的所提出的结构。(b) 残差策略，其输入结构与我们的方法相同，但输出一个残差项，该残差项叠加到参考电机位置 [34, 70]上。(c) 仅长历史策略，该策略仅能访问长期输入输出历史（我们仍按照Peng等人 [48]的建议，向基础网络提供机器人即时反馈）。(d) 仅短历史策略，该策略仅提供短期输入输出历史 [37]。我们还与RMA [31]/教师-学生 [34] 训练策略进行了比较，其中(e)专家策略能够访问特权环境信息（表III），首先通过强化学习进行训练，随后通过监督学习用于训练(f) RMA（学生）策略。RMA还可以通过强化学习使用(g) A-RMA [32] 进行进一步微调。虽然原始RMA [31] 或TS [34], 中不包含短输入输出历史，但在此基准测试中将其纳入以实现公平比较。如果模块的参数未更新，则将其着色。虚线表示参数被复制。

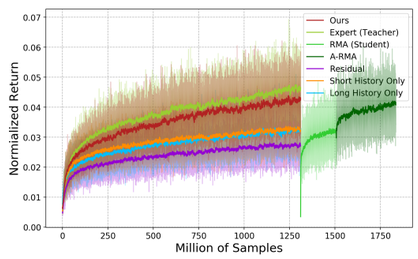


图5：不同策略结构在阶段3（带动力学随机化的多目标训练）中训练的学习曲线基准测试。曲线为使用3个随机种子训练的平均归一化回报，彩色区域覆盖了不同种子间的最小值和最大值。归一化回报通过回报除以最大回合长度计算，范围在 $[0, 1]$内。我们的方法表现出与用于监督RMAs且可访问特权环境参数的专家策略相似的性能。A-RMA表现出次优性能，但与所提方法相比需要显著更多的样本，其次是RMA。仅短历史和仅长历史的策略表现出相似的学习性能，但在回报方面略逊于RMA。残差策略表现最差，因为添加到策略动作中的参考运动阻止了智能体探索更多样化的机动动作。

A. 基线

为回答第一个问题，我们将所提策略架构与图4所示的几个基线模型进行了基准测试。所有策略均使用多目标进行训练，即，跳跃至不同着陆位置和转向方向（无海拔变化），采用图2所示的训练示意图，并使用3个不同的随机种子进行训练。基线模型的详细信息见附录C。

为回答第二个问题，我们使用所提策略结构获得了两个单目标策略，具体如下：

- **单目标**：一个在单一原地跳跃任务上训练的策略，并进行了表III所列的广泛动力学随机化。
- **带扰动的单目标**：一个与单目标策略相似的策略，但额外训练了施加在机器人骨盆上的随机扰动扳手（六自由度）。外力和扭矩从 $[-20\text{N}\ -5\text{Nm}]$ 到 $[20\text{N},\ 5\text{Nm}]$ 范围内均匀采样，并在随机时间间隔（从 $[0.1, 2.0]$ 秒起）内施加于机器人骨盆。

我们基于两个指标对这些基线进行比较：（1）第VI-B节中的学习性能；（2）在第VI-C节中对训练分布之外的动力学参数的泛化能力。这两个指标对于仿真到现实迁移至关重要，因为前者展示了策略在训练过程中的表现，而后者则评估了策略对环境变化的鲁棒性——这些变化在训练中未被考虑，正如仿真到现实迁移中可能出现的情况。

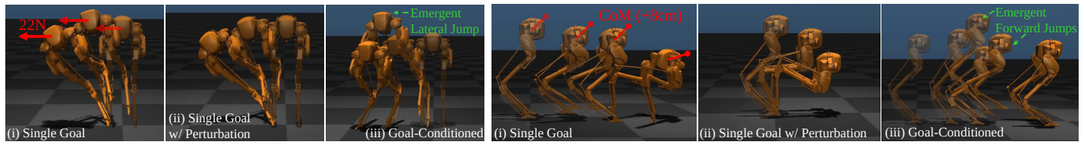
B. 策略结构选择

图5展示了在阶段3（结合动力学随机化的多目标学习）中，使用我们的策略结构和基线得到的学习曲线。早期训练阶段（阶段1中学习单一任务和阶段2中学习多任务）的学习曲线见附录D中的图10。每个训练阶段均使用相同的超参数和奖励函数。

根据图5（及图10），紫色曲线所示的残差结构在所有训练阶段中表现出最差的学习性能。原因在于我们提供的参考运动是动态不可行动画，这可能导致机器人花费更多精力学习纠正这些默认运动，并阻止其探索更多样化的轨迹以及推断超出参考运动范围的运动。

仅使用短历史（橙色曲线）和仅使用长历史（蓝色曲线）的基线显示出相似的学习性能。但如果我们通过为策略提供长历史编码器和对短历史的直接访问来将两者结合，从而形成我们的方法，学习性能可以得到大幅提升，如图5中的红色曲线所示。这表明，仅为策略提供长历史是不够的，因为机器人可能需要即时反馈，而这种反馈可能被长历史编码器隐藏。为策略提供对短历史的直接访问可以解决这一问题，智能体能够学会利用这两种信息。

备注3：我们注意到，有其他工作使用带有LSTM的RNNs [48, 58, 59] 或TCN [34] 来编码长期输入输出历史。我们假设，为策略提供



(a) With Consistent Unknown Lateral Perturbation Force (b) With Errors in Center of Mass Positions of All Links

Fig. 6: Robustness comparison among three policies which are: (i) trained with a single task (jumping in place) with dynamics randomization, (ii) trained with a single task with dynamics randomization and random perturbation, and (iii) trained with multiple tasks with dynamics randomization but without random perturbation (proposed). The testing scenarios are outside the training setting for all three policies. The single-goal policies fail to stabilize the robot, even the one trained with extensive perturbations. The goal-conditioned policy which is trained with diverse jumping tasks but without perturbation succeeds to stabilize the robot by exploiting the learned skills. The goal-conditioned policy is able to deviate from the commands (jumping in place) and utilize a lateral jump to stay robust to the lateral external force and two forward jumps to adapt to the forward CoM offset.

direct access to the *short history* is not limited to the 1D CNN encoder but also to other neural network structures that capture temporal information such as TCN, LSTM, GRU, and Transformer. We choose 1D CNN in this work because it is easier to train.

The comparison between our method and RMA/Teacher-Student (TS) policies (green curves) is also interesting. During the training of the goal-conditioned policy with dynamics randomization, our method only shows a little degradation compared to the expert policy. This actually showcases the advantages of our method because it can be zero-shot transferred to the real world while the expert policy that requires privileged information cannot. After training the expert policy, RMA and A-RMA have trained with 3k iterations and 5k iterations respectively, as shown in Fig. 5. We found that RMA has a large degradation compared to the expert policy due to the regression loss, and A-RMA is necessary to finetune the base policy in order to further improve the return. RMA shows a better return than the policy with only short history or only long history, which is aligned with the finding from previous work [31, 32]. However, even after A-RMA converged, the return is a bit worse than our method, while RMA and A-RMA require additional training and significantly more samples.

Remark 4: The original implementation of RMA [31] or TS [34] only provides the robot's very last I/O pair [31] or last state feedback [34] besides the long-term history encoder. In the implementation of RMA/TS in Fig. 4, we added the short-term I/O history, which can improve the learning performance in order to have a fair comparison. Furthermore, the long-term I/O history encoder used in RMA and A-RMA is the same as the one used in the proposed method, which shows a better learning performance than the original encoder proposed in [31, 32] as shown in Fig. 11b.

Summary of the Result: By the ablation study above, we can summarize three factors that can improve the learning performance in our case for dynamic locomotion control: (1) using desired motion positions as the action space (in contrast to the residual), (2) providing the policy with direct access to the short-term I/O history in addition to a long-term robot's I/O history, and (3) training the policy in an end-to-end way instead of separating the training process into teacher and student. This combination leads to our proposed method.

C. Advantages of the Verstiale Policy

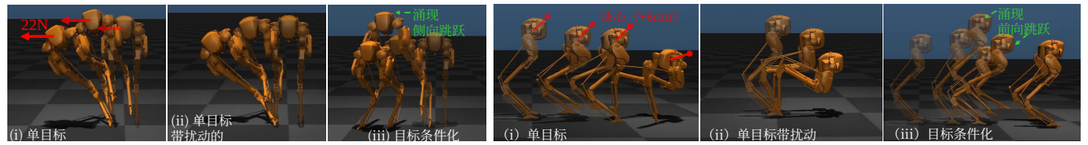
In order to validate the advantages brought by multi-goal training, we further compare our goal-conditioned policy with the single-goal policies. These two single-goal policies are trained with the same amount of samples with dynamics randomization as the proposed one, whose learning curves are recorded in Fig. 11a. During the test in simulation, we command the robot to perform an in-place jump in an environment that the robot has not been trained on. As presented in Fig. 6, we conducted two tests where (1) a consistent lateral perturbation force is applied on the robot pelvis, and (2) the CoM of *all* links are set to be $+8$ cm off from the default position in all dimensions, while other dynamics parameters are set to the default values.

During these two tests, both of the single-goal policies fail to control the robot, while the goal-conditioned policy succeeded to stabilize the robot and perform a jump. Specifically, the policies trained with a single goal directly fail during standing, even in the case where one is trained with extensive external perturbations that "force" the robot to explore more maneuvers by perturbing it from a nominal jump. On the contrary, the policy trained with multiple goals, such as jumping forwards and lateral, without perturbations during training, is able to generalize the learned tasks, exploit them to stabilize the robot, and pick the best jumping maneuver even if it is not commanded. For example, while being commanded to jump in place, the goal-conditioned policy utilizes a lateral jump that it learned to stabilize the robot with the presence of lateral force (Fig. 6a(iii)) and two emergent forward jumps to adapt to the CoM errors in the forward direction (Fig. 6b(iii)). Such a benchmark highlights the advantages of learning with multiple tasks which makes the policy more robust.

Having conducted an extensive ablation study in simulation, we show that the proposed policy structure and multi-goal training significantly improve the robustness of the policy over other policy structures or single-goal policies.

VII. Experiments

We now deploy the goal-conditioned policies obtained in simulation, the *flat-ground policy* that is trained on different goals to jump to various locations and turning directions, and



(a) 存在一致未知横向扰动力 (b) 存在所有连杆质心位置误差

图6：三种策略的鲁棒性比较，分别为：(i) 使用动力学随机化训练的单一任务（原地跳跃）策略，(ii) 使用动力学随机化和随机扰动训练的单一任务策略，以及 (iii) 使用多任务训练、包含动力学随机化但不含随机扰动的策略（本文提出的方法）。所有三种策略的测试场景均超出其训练设置。单目标策略无法稳定机器人，即使是经过大量扰动训练的策略也未能成功。而通过多样化跳跃任务训练、但不含随机扰动的目标条件策略，则成功利用已学技能稳定了机器人。该目标条件策略能够偏离指令（原地跳跃），利用侧向跳跃来抵抗横向外力，并通过两次前向跳跃来适应前向质心偏移。

对短历史的<直接访问>并不局限于一维卷积神经网络编码器，也适用于其他能够捕捉时序信息的神经网络结构，如TCN、LSTM、GRU和Transformer。我们在本工作中选择一维卷积神经网络，因为它更易于训练。

我们的方法与RMA/教师-学生（TS）策略（绿色曲线）之间的对比也颇具启发性。在采用动力学随机化训练目标条件策略的过程中，我们的方法与专家策略相比仅表现出轻微的性能下降。这实际上凸显了我们方法的优势，因为它可以零样本迁移到真实世界，而需要特权信息的专家策略则无法做到。在训练完专家策略后，RMA和A-RMA分别经过3000次和5000次迭代训练，如图5所示。我们发现，由于回归损失的存在，RMA相比专家策略出现了较大的性能下降，而A-RMA则需要对基础策略进行微调以进一步提升回报。RMA的回报优于仅使用短历史或仅使用长历史的策略，这与先前研究 [31, 32]的发现一致。然而，即使在A-RMA收敛之后，其回报仍略逊于我们的方法，而RMA和A-RMA还需要额外的训练和显著更多的样本。

备注*4*：RMA [31] 或TS [34] 的原始实现仅向机器人提供其最近的输入/输出对 [31] 或最后的状态反馈 [34]，此外还有长期历史编码器。在图4的RMA/TS实现中，我们添加了短期输入输出历史，这可以提升学习性能，以便进行公平比较。此外，RMA和A-RMA中使用的长期输入输出历史编码器与所提方法中使用的相同，其表现出的学习性能优于 [31, 32]中提出的原始编码器，如图11b所示。

结果总结：通过上述消融研究，我们可以总结出三个能够提升我们动态运动控制案例中学习性能的因素：(1) 使用期望运动位置作为动作空间（而非残差），(2) 除了长期机器人输入输出历史外，还向策略提供对短期输入/输出历史的直接访问，以及 (3) 以端到端方式训练策略，而非将训练过程分为教师和学生。这种组合构成了我们提出的方法。

C. Verstiale Policy的优势

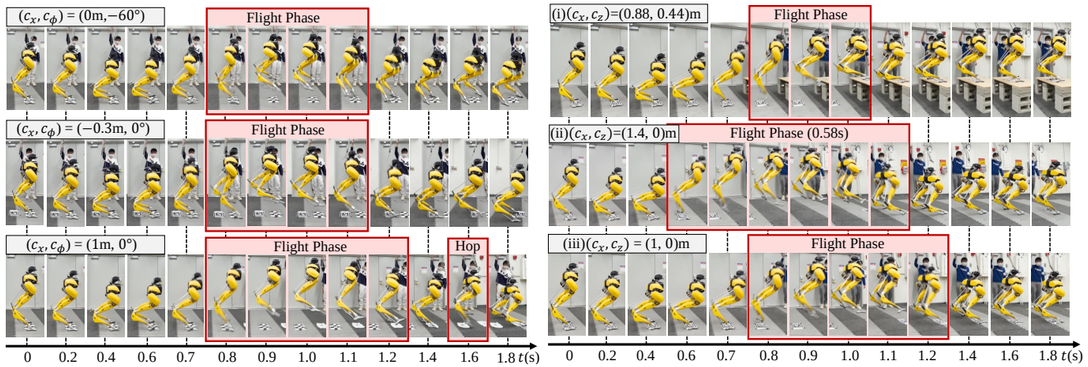
为了验证多目标训练带来的优势，我们进一步将我们的目标条件策略与单目标策略进行了比较。这两个单目标策略与所提出的策略使用相同数量的样本进行训练，并采用了动力学随机化，其学习曲线记录在图11a中。在仿真测试中，我们命令机器人在一个未经训练的环境中进行原地跳跃。如图6所示，我们进行了两项测试：(1) 对机器人骨盆施加持续的横向扰动力，以及 (2) 将所有连杆的质心设置为在三个维度上偏离默认位置 $+8$ 厘米，而其他动力学参数则设置为默认值。

在这两项测试中，两个单目标策略均未能控制机器人，而目标条件策略成功稳定了机器人并执行了跳跃。具体而言，使用单目标训练的策略在站立阶段直接失败，即使其中一个策略通过大量外部扰动进行训练，这些扰动通过干扰标称跳跃来“迫使”机器人探索更多动作。相反，使用多目标（如向前跳跃和侧向跳跃）训练的策略，在训练过程中没有扰动，能够泛化所学任务，利用它们稳定机器人，并选择最佳跳跃动作，即使该动作并非指令所要求。例如，当被指令原地跳跃时，目标条件策略利用其学到的侧向跳跃来稳定机器人以应对侧向力（图6a(iii)），并涌现出两次前向跳跃以适应前向方向上的质心误差（图6b(iii)）。这一基准测试凸显了多任务学习的优势，使策略更加鲁棒。

通过在仿真中进行广泛的消融研究，我们表明所提策略结构和多目标训练相比其他策略结构或单目标策略，显著提升了策略的鲁棒性。

VII. 实验

我们现在将在仿真中训练得到的目标条件策略部署到Cassie硬件上，包括针对不同目标训练、可跳跃至不同位置和转向方向的平地策略，以及



(a) Different Jumps using the Flat-ground Policy

(b) Different Jumps using the Discrete-terrain Policy

Fig. 7: Snapshots of Cassie performing different jumps using the proposed goal-conditioned policies. The snapshots are aligned with timestamps. The tags in the figures indicate the given landing targets. (a) Using a single policy that is specialized on flat ground, the robot is able to (i) jump in place while turning to $-60°$, (ii) jump 0.3 m backward, and (iii) jump 1 m forward, respectively. During the 1 m jump, the robot utilizes a forward hop to reach the goal after it lands on 0.5 m after the first jump. (b) The robot utilizes a single discrete-terrain policy to jump to different locations and elevations. The single policy can change the contact plan for different tasks. For example, the flight phase of the 1.4 m forward jump (ii) is the longest while being the shortest when the robot jumps onto the 0.44 m high elevation (i). The robot can land at the target (tag) with insignificant errors among all of these jumps without global position feedback.

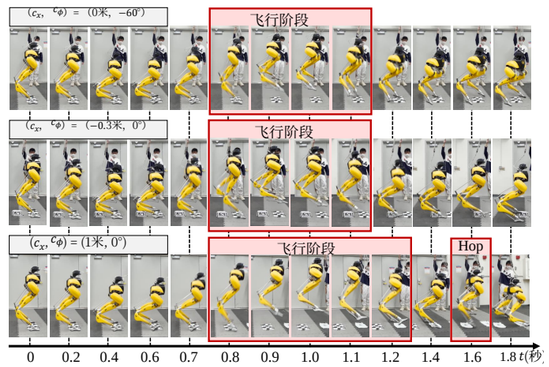
the *discrete-terrain policy* that is specialized in jumping to variable locations and elevations, on the hardware of Cassie. As shown in Fig. 1, both policies can successfully control the robot in the real world, without finetuning.

Besides the ability to succeed in sim-to-real, in this section, we aim to validate two hypotheses: 1) whether the policy trained in simulation can complete the same task in the real world, and 2) whether the goal-conditioned policy is still able to exploit the learned tasks to stabilize the robot after being transferred to the real world. The experiments can be best seen in the accompanying video (https://youtu.be/aAPSZ2QFB-E). Please note that in all of the experiments, the robot does not have global position feedback, *i.e.*, once it starts to move, it does not know the distance to the landing target nor the distance to the ground.

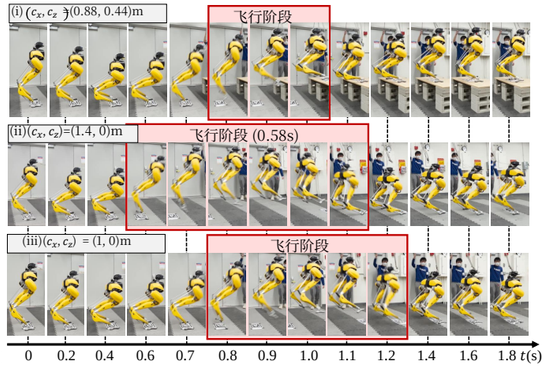
A. Task Completion in the Real World

We first test the flat-ground policy on the robot in three distinct tasks: jumping in place while turning to negative 60 degrees, jumping 0.3 m backward, and jumping forward to land at a 1 m ahead target. As recorded in Fig. 7a, controlled by this goal-conditioned policy, the robot is able to complete all three tasks. For example, during the turning task, the robot rotates to $-55°$ while jumping in the air, and lands at the same place where it took off (marked by a tag on the ground in Fig. 7a(i)). During the backward jump, different from the previous task, the robot leans backward before taking off (0.6-0.7 sec in Fig. 7a(ii)), and lands accurately at the target tag on the ground. To jump 1 m forward, the robot adopts a different maneuver where it leans forward before the flight phase and pushes itself off the ground with a larger strength, which results in a longer flight phase and travel distance than the previous two tasks. We also observe that the robot first lands at a 0.5 m landmark, but quickly conducts a forward hop when legs touch the ground (1.6 sec in Fig. 7a(iii)), and lands at the 1 m target tag in the last. We note that such a consecutive jumping maneuver does not happen during the same task in the simulation with the robot's nominal dynamics model. Such an experiment highlights two favorable features of the proposed policy: it can (1) adapt to different system dynamics (from sim to real) and (2) deviate from the reference motion and utilize multiple contacts to complete the given task (jumping to the target).

We then validate the discrete-terrain policy with three tasks: Jumping 1 m ahead, 1.4 m ahead, and to a target that is 0.88 m ahead and 0.44 m above the ground, as presented in Fig. 7b. It shows the capacity to control the robot to jump over a distance/elevation to land on the given target accurately. We notice that the policy is able to adjust the robot's maneuvers/contact plan to jump over different distances/elevations. For example, compared to the 1 m jump (Fig. 7b(iii)), the robot takes off earlier while landing later during the 1.4 m jump (Fig. 7b (ii)). Such a change is reasonable as the robot needs a longer flight phase/larger take-off velocity in order to land at a farther location. Furthermore, when the robot is commanded to jump onto a 0.44 m table, the policy jumps more vertically (0.8 sec in Fig. 7b(i)) compared to the 1.4 m jump (0.6 sec in Fig. 7b(ii)) at the beginning of the flight phase while lifting the robot legs much higher in order to jump higher. Note that the robot makes contact with the platform much earlier than the other jumps on the ground, but this single policy is still able to stabilize the robot with different landing events. Furthermore, all these three experiments show that the robot controlled by the proposed policy can land accurately on the given target (land on the tags in Fig. 7b), which is challenging. Because the robot's motion is ballistic in the flight phase and a small



(a) 使用平地策略的不同跳跃



(b) 使用离散地形策略的不同跳跃

图7：Cassie使用所提出的目标条件策略执行不同跳跃的快照。快照按时间戳对齐。图中的标签表示给定的着陆目标。(a) 使用专门针对平坦地面的单一策略，机器人能够 (i) 原地跳跃并转向 $-60°$，(ii) 向后跳跃0.3米，以及 (iii) 向前跳跃1米。在1米跳跃过程中，机器人在第一次跳跃后着陆于0.5米处，利用前跳到达目标。(b) 机器人使用单一的离散地形策略跳跃到不同位置和抬升高度。该单一策略可以针对不同任务改变接触计划。例如，1.4米向前跳跃 (ii) 的飞行阶段最长，而机器人跳上0.44米高抬升 (i) 时飞行阶段最短。在所有这些跳跃中，机器人无需全局位置反馈即可着陆于目标（标签）处，且误差极小。

专门用于跳跃至不同位置和抬升高度的离散地形策略。如图1所示，两种策略均能在无需微调的情况下，成功在真实世界中控制机器人。

除了具备成功实现仿真到现实迁移的能力外，本节旨在验证两个假设：1）在仿真中训练的策略是否能在真实世界中完成相同任务；2）目标条件策略在迁移到真实世界后，是否仍能利用已学任务来稳定机器人。相关实验可最佳地通过随附视频（https://youtu.be/aAPSZ2QFB-E）观看。请注意，在所有实验中，机器人不具备全局位置反馈，即，一旦开始移动，它便不知道与着陆目标的距离，也不知道与地面的距离。

A. 真实世界中的任务完成

我们首先在机器人上测试了平地策略在三个不同任务中的表现：原地跳跃并转向负60度、向后跳跃0.3米，以及向前跳跃至前方1米处的目标。如图7a所示，在该目标条件策略的控制下，机器人能够完成所有三项任务。例如，在转向任务中，机器人在空中跳跃时旋转至 $-55°$，并降落在起跳点（图7a(i)中地面标记处）。在向后跳跃任务中，与前一项任务不同，机器人在起跳前身体向后倾斜（图7a(ii)中0.6-0.7秒），并准确降落在目标标签处。为了向前跳跃1米，机器人采用了不同的动作：在飞行阶段前身体前倾，并以更大的力量蹬地，这使得飞行阶段和行进距离均长于前两项任务。我们还观察到，机器人首先降落在0.5米标记处，但随后在腿部触地时迅速进行了一次前跳（图7a(iii)中1.6秒），最终降落在1米目标标签处。我们注意到，这种连续跳跃动作在使用机器人标称动力学模型的仿真中并未出现。该实验凸显了所提策略的两个优点：(1) 能够适应不同的系统动力学（从仿真到真实），以及 (2) 能够偏离参考运动并利用多次接触来完成给定任务（跳跃至目标）。

随后，我们通过三项任务验证了离散地形策略：向前跳跃1米、1.4米，以及跳跃至一个位于前方0.88米、离地0.44米的目标，如图7b所示。这展示了该策略控制机器人跨越一定距离/抬升高度并精确着陆于给定目标的能力。我们注意到，该策略能够调整机器人的机动方式/接触计划，以跨越不同的距离/抬升高度。例如，与1米跳跃（图7b(iii)）相比，机器人在1.4米跳跃（图7b(ii)）中起飞更早而着陆更晚。这种变化是合理的，因为机器人需要更长的飞行阶段/更大的起飞速度才能着陆于更远的位置。此外，当指令要求机器人跳上0.44米高的平台时，与1.4米跳跃（飞行阶段开始时为0.6秒，图7b(ii)）相比，该策略在飞行阶段开始时（0.8秒，图7b(i)）的跳跃更垂直，同时将机器人腿部抬得更高以实现更高跳跃。值得注意的是，机器人接触平台的时间远早于其他地面跳跃，但这一单一策略仍能通过不同的着陆事件稳定机器人。此外，这三项实验均表明，由所提策略控制的机器人能够精确着陆于给定目标（着陆于图7b中的标签上），这极具挑战性。因为机器人在飞行阶段的运动是弹道式的，且起飞时的微小

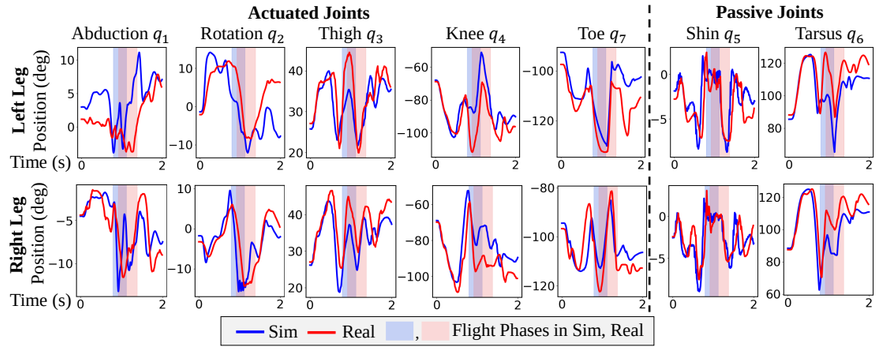


Fig. 8: The profiles of the robot's joint positions when it is commanded to jump and turn -60° in place in simulation and the real world. We observe a large deviation of the joint profiles between sim and real, *e.g.*. the tarsus joints which are passive and driven by leaf springs show a significant difference during the sim-to-real transfer. Moreover, the flight phase in the real world is delayed compared with the one in the sim. Such errors highlight a big sim-to-real gap but our policy is robust to this and succeeds in controlling the robot to the given target.

error during taking-off may result in a large deviation from the landing target. In these experiments, the proposed policy is able to adapt to the dynamics of the robot hardware, adjust the robot's pose during the take-off, and accelerate to a velocity that can land the robot on the target.

Remark 5: During the long jump (like Fig. 7b(ii)), the robot leans its body forward at a large angle when it is pushing off from the ground and swings the legs forward during descent, and rotates its body forward w.r.t. contact points after it lands. Such a maneuver is very close to what we observed when a human athlete performs a standing jump [44, Fig. 1A]. Similar to humans, our robot's long jump skill is also learned during training, which is very different from the jumping-in-place reference motion we provided.

B. Sim-to-Real Gap

In order to further understand the difficulty to succeed in robot jumping experiments in Fig. 7, we take a close look at the sim-to-real gap. We record the robot's joint position profiles during a jump in the simulation with the robot's nominal dynamics parameters and on the robot's hardware. The profiles for a turning task (-60°, Fig. 7a(i)) using the flat-ground policy is presented in Fig. 8. According to the recorded profiles, the robot's actual joint position has a large deviation between the simulation (blue curves) and the real world (red curves). For example, the maximum error on the tarsus joint position q_6 between sim and real is over 0.35 rad, which largely affects the robot's dynamics considering this joint is not actuated and is driven by a leaf spring whose nominal stiffness is 1250 Nm/rad. A similar deviation is observed in other joints, such as rotation joints q_2, thigh joints q_3 and knee joints q_4, which play a critical role during a jump and turning, and in other experiments using the discrete-terrain policy as recorded in Fig. 12. Such a discrepancy highlights the huge gap between the simulation and the real world but also showcases that, despite such a large gap, our methodology introduced in Sec. IV is able to stay robust and succeed in controlling the robot to accomplish the task.

C. Diverse and Robust Maneuvers by the Goal-Conditioned Policy

In order to push the limits of the proposed control policies, we further conduct more dynamic jumping experiments as presented in Fig. 9 and Fig. 13. As shown in Fig. 9a, using the single flat-ground policy, the robot performs a large repertoire of dynamic jumping maneuvers, such as jumping in place (Fig. 9a(i)), jumping to lateral (Fig. 9a(iii)), and multi-axes jumps such as blending lateral and forward jumps (Fig. 9a(iv)) and forward, lateral and turning (Fig. 9a(v)). In these multi-axes jumps, the robot demonstrates more complex maneuvers. For example, the robot leans in the lateral direction while jumping forward and turning to land on the target that is 0.5 m ahead, 0.2 m to robot's left, and turned -45°, as shown in Fig. 9a(v). During some challenging tasks, the robot is aware to utilize small hops to adjust its body pose after it lands with unstable states, such as demonstrated in Fig. 9a(iv)(v).

Moreover, in order to test the robustness of the policy, we applied a backward perturbation force on the robot's pelvis at its apex jumping height, as shown in Fig. 9a(ii). Due to such a perturbation, the robot leans backward during descending, and both of its toes pitch up after it lands, which makes the robot underactuated w.r.t contact points. However, the robot quickly exerts a backward hop, which is learned during the multi-goal training, after it lands. By this hop, the robot can adjust its body pose during the flight phase and then land stably afterward. The goal we gave to the robot in this test is to jump in place and it is interesting to see that the robot deviates from it in order to recover from falling over.

Remark 6: The robot, controlled by the proposed jumping policy, shows the ability to not rely on the pre-defined contact plan and can break the contact after it lands and make contact again when it needs to utilize impacts to stabilize itself. Such a capability is similar to contact implicit trajectory optimization [8, 14, 33, 52, 77]. While such optimization schemes still need to be computed offline for legged robots, our work achieves this online.

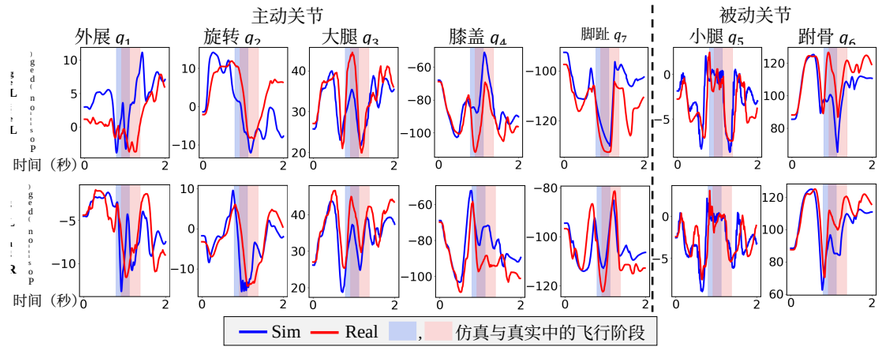


图8：当机器人在仿真和真实世界中执行原地跳跃并转向 -60° 时，其关节位置的变化曲线。我们观察到仿真与真实环境之间关节曲线的显著偏差，例如，由板簧驱动的被动跗关节在仿真到现实迁移过程中表现出明显差异。此外，真实世界中的飞行阶段相比仿真有所延迟。这些误差凸显了较大的仿真到现实差距，但我们的策略对此具有鲁棒性，并成功控制机器人到达指定目标。

误差可能导致与着陆目标的巨大偏差。在这些实验中，所提策略能够适应机器人硬件的动力学特性，在起飞过程中调整机器人的姿态，并加速至能使机器人着陆于目标的速度。

备注*5*：在跳远过程中（如图7b(ii)所示），机器人蹬地时身体以较大角度向前倾斜，下落时双腿向前摆动，并在落地后相对于接触点向前旋转身体。这种动作与我们观察到的人类运动员进行立定跳远 [44, 图1A]时的表现非常接近。与人类相似，我们机器人的跳远技能也是在训练中习得的，这与我们提供的原地跳跃参考运动截然不同。

B. 仿真到现实差距

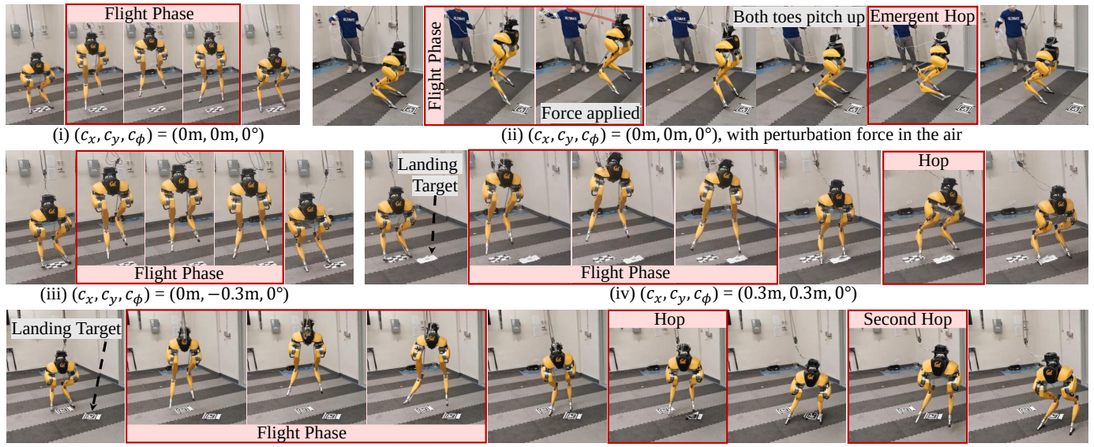
为了进一步理解图7中机器人跳跃实验成功的难度，我们仔细审视了仿真到现实差距。我们记录了机器人在标称动力学参数下的仿真中以及在实际硬件上进行跳跃时的关节位置曲线。图8展示了使用平地策略进行转向任务（-60°，图7a(i)）时的曲线。根据记录的曲线，机器人的实际关节位置在仿真（蓝色曲线）与真实世界（红色曲线）之间存在较大偏差。例如，跗骨关节位置 q_6 在仿真与现实之间的最大误差超过0.35弧度，考虑到该关节是非驱动的，由标称刚度为1250牛米/弧度的板簧驱动，这极大地影响了机器人的动力学。在其他关节，如旋转关节 q_2、大腿关节 q_3 和膝关节 q_4（这些关节在跳跃和转向中起关键作用）以及使用离散地形策略的其他实验（如图12所记录）中也观察到了类似的偏差。这种差异凸显了仿真与真实世界之间的巨大鸿沟，但也表明，尽管存在如此大的差距，我们在第四节中介绍的方法仍能保持鲁棒性，并成功控制机器人完成任务。

C. 目标条件策略实现多样且鲁棒的动作

为了进一步挑战所提出的控制策略的极限，我们进行了更多动态跳跃实验，如图9和图13所示。如图9a所示，使用单一平地策略，机器人执行了多种动态跳跃动作，例如原地跳跃（图9a(i)）、侧向跳跃（图9a(iii)）以及多轴跳跃，如混合侧向与前向跳跃（图9a(iv)）和前向、侧向与转向（图9a(v)）。在这些多轴跳跃中，机器人展示了更复杂的动作。例如，如图9a(v)所示，机器人在向前跳跃并转向以落在前方0.5米、机器人左侧0.2米且转向 -45° 的目标上时，会向侧向倾斜。在一些具有挑战性的任务中，机器人在以不稳定状态落地后，会利用小幅跳跃来调整其身体姿态，如图9a(iv)(v)所示。

此外，为了测试策略的鲁棒性，我们在机器人骨盆处于跳跃最高点时施加了一个向后扰动力，如图9a(ii)所示。由于这种扰动，机器人在下降过程中会向后倾斜，落地后其两个脚趾均上仰，这使得机器人相对于接触点处于欠驱动状态。然而，机器人在落地后迅速执行了一个向后跳跃动作，该动作是在多目标训练中习得的。通过这次跳跃，机器人能够在飞行阶段调整其身体姿态，随后稳定落地。在此测试中，我们给机器人的目标是原地跳跃，有趣的是，机器人为了从摔倒中恢复而偏离了这一目标。

备注*6*：由所提出的跳跃策略控制的机器人，展现出不依赖预定义接触计划的能力，能够在落地后断开接触，并在需要利用冲击来稳定自身时重新建立接触。这种能力类似于接触隐式轨迹优化 [8, 14, 33, 52, 77]。尽管此类优化方案仍需为腿式机器人进行离线计算，但我们的工作实现了在线完成。

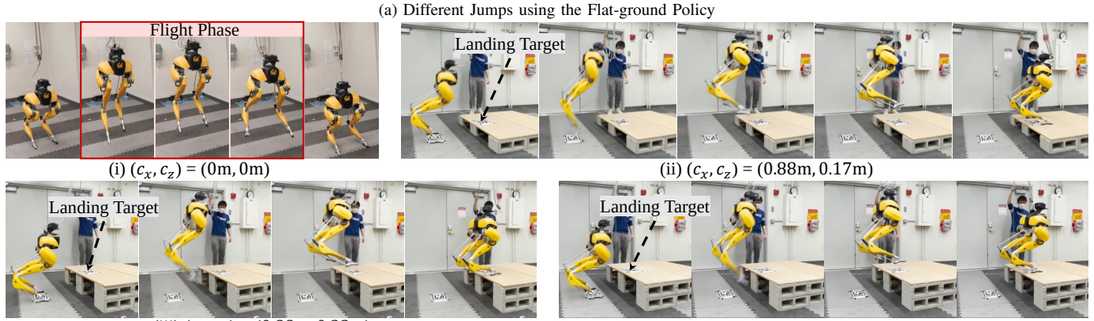


(i) $(c_x, c_y, c_\phi) = (0\text{m}, 0\text{m}, 0°)$ (ii) $(c_x, c_y, c_\phi) = (0\text{m}, 0\text{m}, 0°)$, with perturbation force in the air

(iii) $(c_x, c_y, c_\phi) = (0\text{m}, -0.3\text{m}, 0°)$ (iv) $(c_x, c_y, c_\phi) = (0.3\text{m}, 0.3\text{m}, 0°)$

(v) $(c_x, c_y, c_\phi) = (0.5\text{m}, 0.2\text{m}, -45°)$

(a) Different Jumps using the Flat-ground Policy



(i) $(c_x, c_z) = (0\text{m}, 0\text{m})$ (ii) $(c_x, c_z) = (0.88\text{m}, 0.17\text{m})$

(iii) $(c_x, c_z) = (0.88\text{m}, 0.32\text{m})$ (iv) $(c_x, c_z) = (0.64\text{m}, 0.32\text{m})$

(b) Different Jumps using the Discrete-terrain Policy

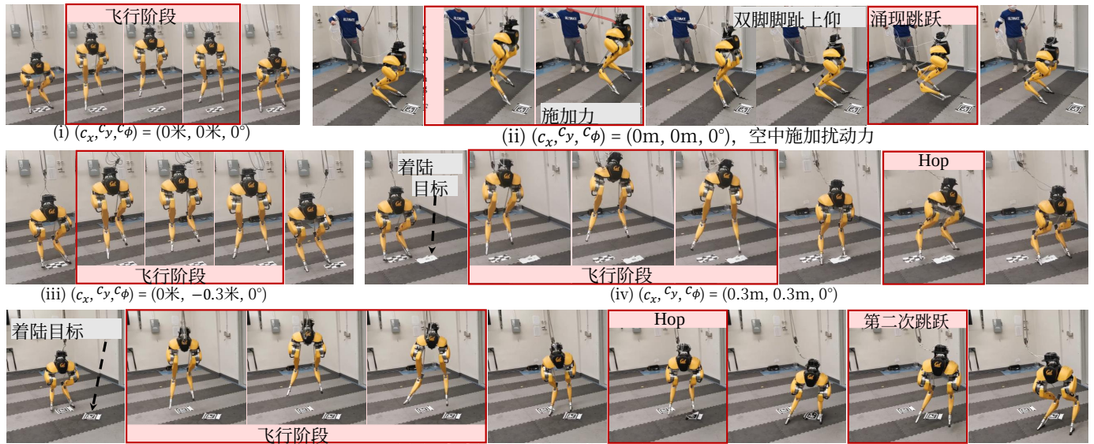
Fig. 9: Snapshots of various dynamic jumps performed by Cassie using the proposed policies. (a) The robot is able to perform a large repertoire of multi-axes jumps on flat ground. It shows the ability to stabilize the robot from a backward external perturbation (ii) by deviating from the commanded in-place jump and exploiting the maneuvers learned from backward jumping tasks. The robot also leverages emergent hops after landing to stabilize it from a huge impact force, while being commanded to stand, like (iv) and (v). (b) Using a single discrete-terrain policy, the robot can not only jump in place (i) but also jump to different locations with different elevations (ii) (iii) (iv).

In the additional testing of the discrete-terrain policy demonstrated in Fig. 9b, the robot shows the ability to accurately land on different given targets. While the changes in the commanded distance and elevation are relatively small, the policy still demonstrates the ability to adjust the robot's take-off maneuvers in order to jump to the given targets.

VIII. Discussion of Design Choices for RL-based Legged Locomotion Control

In this section, we discuss the lessons learned through the development of jumping controllers for bipedal robots using RL. We hope this can provide useful insights for future endeavors on applying RL for legged locomotion.

Short-term history complements the long-term history: Providing a long-term history of the robot's input (policy's action) and/or output (measurement feedback) has been used in many prior efforts in RL-based robotic controls [31, 34, 48, 58, 59]. While these prior systems show the advantages of using a long-term history over only the current state feedback [48, 59], the advantages over a short history [37, 43, 49] were not investigated. In this work, we demonstrate that incorporating *both* short-term history and long-term history can be beneficial. The ablation study in Fig. 5 shows that providing only the long-term I/O history (Fig. 4c) may not be sufficient, even when combined with observations of the robot's current state besides the history encoder, which is analogous to [31, 34, 48]. The learning performance of such a method shows no significant difference between the MLP policy with only short-term history (Fig. 4d), as shown in Fig. 5. Our architecture (Fig. 4a) exhibits better learning performance because it has direct access to a short-term history while also having a long-term history encoder. During real-

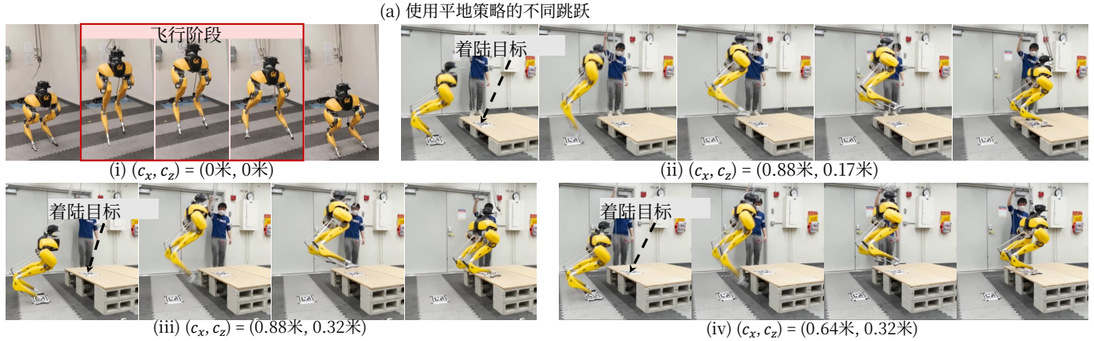


(i) $(c_x, c_y, c_\phi) = (0$米, 0米, $0°)$ (ii) $(c_x, c_y, c_\phi) = (0\text{m}, 0\text{m}, 0°)$，空中施加扰动力

(iii) $(c_x, c_y, c_\phi) = (0$米, -0.3米, $0°)$ (iv) $(c_x, c_y, c_\phi) = (0.3\text{m}, 0.3\text{m}, 0°)$

(v) $(c_x, c_y, c_\phi) = (0.5$米, 0.2米, $-45°)$

(a) 使用平地策略的不同跳跃



(i) $(c_x, c_z) = (0$米, 0米) (ii) $(c_x, c_z) = (0.88$米, 0.17米)

(iii) $(c_x, c_z) = (0.88$米, 0.32米) (iv) $(c_x, c_z) = (0.64$米, 0.32米)

(b) 使用离散地形策略的不同跳跃

图9：Cassie使用所提策略执行各种动态跳跃的快照。(a) 机器人能够在平坦地面上执行多种多轴跳跃。它展示了通过偏离指令的原地跳跃并利用从向后跳跃任务中学到的机动动作，从向后外部扰动（ii）中稳定机器人的能力。机器人还在着陆后利用涌现式弹跳来稳定自身，以应对巨大的冲击力，如（iv）和（v）所示。(b) 使用单一的离散地形策略，机器人不仅可以原地跳跃（i），还可以跳跃到不同高度的不同位置（ii）（iii）（iv）。

在对图9b所示的离散地形策略进行的额外测试中，机器人展现出精准降落在不同给定目标上的能力。尽管指令距离和抬升高度的变化相对较小，该策略仍展现出调整机器人起跳动作以跳跃至给定目标的能力。

VIII. 基于强化学习的腿式运动控制设计选择讨论

在本节中，我们讨论通过使用强化学习开发双足机器人跳跃控制器所获得的经验教训。我们希望这能为未来将强化学习应用于腿部运动的研究提供有益的见解。

短期历史补充了长期历史：提供机器人输入（策略的动作）和/或输出（测量反馈）的长期历史已被用于在以往许多基于强化学习的机器人控制研究中 [31, 34, 48, 58, 59]。尽管这些先前系统展示了使用长期历史相较于仅依赖当前状态反馈的优势 [48, 59]，但相较于短期历史的优势 [37, 43, 49]尚未得到充分研究。在本工作中，我们证明同时纳入短期历史和长期历史是有益的。图5中的消融研究表明，仅提供长期输入输出历史（图4c）可能并不足够，即使结合了历史编码器之外的机器人当前状态观测（这类似于 [31, 34, 48]）也是如此。此类方法的学习性能与仅使用短期历史的多层感知机策略（图4d）相比并无显著差异，如图5所示。我们的架构（图4a）展现出更优的学习性能，因为它既能直接访问短期历史，又具备长期历史编码器。在实时

time control, the robot's recent feedback and policy outputs (I/O) could be more important than the observations that are further back in time. Although it is also part of the long-term history, such recent information can be obfuscated and hard to extract from the compressed latent representation from the long-term history encoder. The short-term history provides the model with direct access to the most recent observations. Therefore, one of the reasons the proposed method shows the best learning performance in Fig. 5 is not the usage of long-term history, but the combination of short-term and long-term history. In addition to jumping, this design decision may also benefit other locomotion skills.

Encode environment parameters or robot's I/O history? Although the proposed architecture (Fig. 4a), with the exception of the introduction of short I/O history, may resemble the architecture of RMA [31] or Teacher/Student (TS) framework [34] which also has a long-term history encoder (Fig. 4e,f), the objective of the temporal encoder in this work is different from those methods [31, 34]. The long I/O history encoder in RMA or TS is to estimate the *human-selected* environment parameters (*e.g.*, floor friction and robot's model parameters) by matching the predicted extrinsics from the teacher policy. The proposed method, in contrast, jointly trains the long I/O history encoder with the base policy and learns to directly utilize the robot's past I/O trajectories for control. The advantage is, the robot's I/O history implicitly contains more information besides environment parameters, such as impact events and contact wrenches. In this way, the robot has more freedom to extract the information from the long I/O history without being restricted to estimating the pre-selected environment parameters. This is the reason that the proposed method shows improvement over RMA/TS, which separates training into different teacher and student stages, and also requires an additional finetuning stage by A-RMA, which requires more training time and data, as shown in Fig. 5. We would like to also note that, RMA/TS methods potentially have benefits when combined with external sensors like vision [2, 43], which the proposed one may not have.

Robustness comes from versatility: We have observed that some of the prior RL-based locomotion controllers on periodic walking skills show highly robust behaviors during real-world deployment, including being robust to external perturbations [37, 41] or change of terrains [31, 34, 43]. For aperiodic dynamic jumping skills studied in this work, the RL-based policy also demonstrates significant robustness such as Fig. 9a(ii). This phenomenon raises an interesting question: where does the robustness come from and how can we improve robustness when we are using RL for legged locomotion control? While there has been little prior work that studies this source of robustness, in this work, we conduct an ablation study in Sec. VI-C. Fig. 6 clearly shows that one source of this robustness stems from multi-goal training: the versatile RL-based policy learned from different jumping tasks is able to generalize the learned task to recover from the unexpected perturbation (Fig. 6a(iii)) or deviation from the nominal trajectory (Fig. 6b(iii)). Such robustness cannot be easily obtained by extensive dynamics randomization when the policy is limited to a single jumping task (Fig. 6a(i), Fig. 6b(i)), even with additional randomization on the external perturbation (Fig. 6a(ii), Fig. 6b(ii)). Such a result suggests that, besides the commonly-used dynamics randomization, diversifying the tasks, like jumping to different targets or walking with different velocities, can further improve the robustness of the RL-based control policy, which is a desirable property during sim-to-real transfer.

IX. Conclusion

In this work, we presented an RL-based system for learning a large variety of highly-dynamic jumping maneuvers on real-world bipedal robots. We formulated the bipedal jumping problem as a parameterized set of tasks, and develop a goal-conditioned policy that is trained in simulation but can then be deployed directly in the real world. In order to tackle the challenging multi-goal learning problem, we utilized a multi-stage training scheme that divides the problem into three sub-problems and addresses each through different training stages. We showcase that by training with multiple goals, the robot is able to generalize the learned tasks to produce robust emergent recovery behaviors from large landing impact forces or unknown perturbations. The robustness acquired through multi-goal training then also facilitates the sim-to-real transfer process, which can not be easily acquired through single-goal training alone. Furthermore, we present a policy architecture that improves learning performance. Our framework enables a real Cassie robot to perform a suite of challenging jumping tasks, such as jumping to different locations, jumping onto different evaluations, and blending multi-axes movements during a jump. A limitation we observe occasionally during some experiments is that the robot oscillates after a jump. This may be due to the challenges of having a single policy for both dynamic jumps and stationary standing. In the future, it will be interesting to combine this goal-conditioned jumping policy with a more sophisticated perception system to traverse complex environments with greater mobility.

Acknowledgments

This work was supported in part by NSF Grant CMMI-1944722 and Canadian Institute for Advanced Research (CIFAR). The authors would like to thank Dr. Ayush Agrawal, Xuxin Cheng, Jiaming Chen, Xiaoyu Huang, Yiming Ni, Lizhi Yang, and Bike Zhang for their gracious help.

References

[1] Bernardo Aceituno-Cabezas, Carlos Mastalli, Hongkai Dai, Michele Focchi, Andreea Radulescu, Darwin G Caldwell, José Cappelletto, Juan C Grieco, Gerardo Fernández-López, and Claudio Semini. Simultaneous contact, gait, and motion planning for robust multilegged locomotion via mixed-integer convex optimization. *IEEE Robotics and Automation Letters*, 3(3):2531–2538, 2017.

控制过程中，机器人近期的反馈和策略输出（输入/输出）可能比更早时间点的观测更为重要。尽管这些近期信息也属于长期历史的一部分，但它们可能被长期历史编码器生成的压缩潜在表示所模糊，难以提取。短期历史则为模型提供了对最新观测的直接访问权限。因此，图5中所示方法之所以展现出最佳学习性能，原因之一并非单纯使用了长期历史，而是短期历史与长期历史的结合。除了跳跃任务外，这一设计决策也可能有益于其他运动技能。

编码环境参数还是机器人的输入/输出历史? 尽管提出的架构（图4a）除了引入了短输入输出历史外，可能与RMA [31] 或教师/学生（TS）框架 [34] 的架构相似（后者也包含长期历史编码器，如图4e、f所示），但本工作中时间编码器的目标与这些方法不同 [31, 34]。RMA或TS中的长输入输出历史编码器旨在通过匹配教师策略预测的外部因素来估计人工选择的环境参数（例如，地面摩擦和机器人的模型参数）。相比之下，所提出的方法将长输入输出历史编码器与基础策略联合训练，并学习直接利用机器人过去的输入/输出轨迹进行控制。其优势在于，机器人的输入/输出历史除了环境参数外，还隐式包含了更多信息，例如碰撞事件和接触力/力矩。这样，机器人有更多自由度从长输入输出历史中提取信息，而不受限于估计预先选定的环境参数。这就是所提出的方法相较于RMA/TS有所改进的原因——RMA/TS将训练分为不同的教师和学生阶段，并且还需要通过A-RMA进行额外的微调阶段，这需要更多的训练时间和数据，如图5所示。我们还想指出，RMA/TS方法在与视觉等外部传感器结合时可能具有优势 [2, 43],，而所提出的方法可能不具备这一点。

鲁棒性源于多功能性：我们观察到，先前一些基于强化学习的周期性行走技能运动控制器在实际部署中表现出高度鲁棒的行为，包括对外部扰动 [37, 41] 或地形变化 [31, 34, 43] 的鲁棒性。对于本文研究的非周期性动态跳跃技能，基于强化学习的策略也展现出显著的鲁棒性，如图9a(ii)所示。这一现象引发了一个有趣的问题：鲁棒性从何而来，以及在使用强化学习进行腿式运动控制时，我们如何提高鲁棒性？虽然此前很少有研究探讨鲁棒性的这一来源，但本文在第VI-C节进行了消融研究。图6清晰地表明，鲁棒性的一个来源是多目标训练：从不同跳跃任务中学习到的多功能基于强化学习的策略，能够泛化所学任务，以从意外扰动（图6a(iii)）或偏离标称轨迹（图6b(iii)）中恢复。这种鲁棒性无法通过广泛的动力学随机化轻易获得，当策略仅限于单一跳跃任务时（图6a(i)、图6b(i)），即使额外增加了外部扰动的随机化（图6a(ii)、图6b(ii)）也是如此。这一结果表明，除了常用的动力学随机化之外，多样化任务（例如跳跃到不同目标或以不同速度行走）可以进一步提高基于强化学习的控制策略的鲁棒性，这是仿真到现实迁移过程中的一个理想特性。

IX. 结论

在这项工作中，我们提出了一种基于强化学习的系统，用于在真实世界的双足机器人上学习多种高动态跳跃动作。我们将双足跳跃问题形式化为一个参数化任务集，并开发了一种在仿真中训练、但可直接部署到真实世界中的目标条件策略。为了应对具有挑战性的多目标学习问题，我们采用了多阶段训练方案，将问题划分为三个子问题，并通过不同的训练阶段分别处理每个子问题。我们证明，通过多目标训练，机器人能够泛化所学任务，从而在巨大的着陆冲击力或未知扰动下产生鲁棒的涌现恢复行为。通过多目标训练获得的鲁棒性也促进了仿真到现实迁移过程，这是仅通过单目标训练难以实现的。此外，我们提出了一种能够提升学习性能的策略架构。我们的框架使真实的Cassie机器人能够执行一系列具有挑战性的跳跃任务，例如跳跃到不同位置、跳跃到不同评估点，以及在跳跃过程中融合多轴运动。我们在某些实验中偶尔观察到的一个局限性是，机器人在跳跃后会出现振荡。这可能是由于单一策略同时处理动态跳跃和静止站立所带来的挑战。未来，将这种目标条件跳跃策略与更复杂的感知系统相结合，以更高的机动性穿越复杂环境，将是一个有趣的研究方向。

致谢

本研究部分受美国国家科学基金会（NSF）资助项目CMMI-1944722以及加拿大高等研究院（CIFAR）的支持。作者衷心感谢Ayush Agrawal博士、Xuxin Cheng、陈嘉明、黄晓宇、倪一鸣、杨立志和张必可的慷慨帮助。

参考文献

[1]Bernardo Aceituno-Cabezas, Carlos Mastalli, Hongkai Dai, Michele Focchi, Andreea Radulescu, Darwin G Caldwell, Jos´e Cappelletto, Juan C Grieco, Gerardo Fern´andez-L´opez, 和 Claudio Semini。通过混合整数凸优化实现鲁棒多足运动的同步接触、步态与运动规划。*IEEE*机器人与自动化快报，3(3):2531–2538，2017年。

[2] Ananye Agarwal, Ashish Kumar, Jitendra Malik, and Deepak Pathak. Legged locomotion in challenging terrains using egocentric vision. *arXiv preprint arXiv:2211.07638*, 2022.

[3] Ryan Batke, Fangzhou Yu, Jeremy Dao, Jonathan Hurst, Ross L Hatton, Alan Fern, and Kevin Green. Optimizing bipedal maneuvers of single rigid-body models for reinforcement learning. In *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*, pages 714–721, 2022.

[4] Guillaume Bellegarda and Auke Ijspeert. Cpg-rl: Learning central pattern generators for quadruped locomotion. *IEEE Robotics and Automation Letters*, 7(4):12547–12554, 2022.

[5] Guillaume Bellegarda, Yiyu Chen, Zhuochen Liu, and Quan Nguyen. Robust high-speed running for quadruped robots via deep reinforcement learning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 10364–10370, 2022.

[6] Miroslav Bogdanovic, Majid Khadiv, and Ludovic Righetti. Model-free reinforcement learning for robust locomotion using demonstrations from trajectory optimization. *Frontiers in Robotics and AI*, 9, 2022.

[7] Guillermo A Castillo, Bowen Weng, Wei Zhang, and Ayonga Hereid. Robust feedback motion policy design using reinforcement learning on a 3d digit bipedal robot. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5136–5143, 2021.

[8] Iordanis Chatzinikolaidis, Yangwei You, and Zhibin Li. Contact-implicit trajectory optimization using an analytically solvable contact model for locomotion on variable ground. *IEEE Robotics and Automation Letters*, 5(4): 6357–6364, 2020.

[9] Hua Chen, Bingheng Wang, Zejun Hong, Cong Shen, Patrick M Wensing, and Wei Zhang. Underactuated motion planning and control for jumping with wheeled-bipedal robots. *IEEE Robotics and Automation Letters*, 6(2):747–754, 2020.

[10] Hongkai Dai, Andrés Valenzuela, and Russ Tedrake. Whole-body motion planning with centroidal dynamics and full kinematics. In *2014 IEEE-RAS International Conference on Humanoid Robots*, pages 295–302, 2014.

[11] Yanran Ding, Chuanzheng Li, and Hae-Won Park. Single leg dynamic motion planning with mixed-integer convex optimization. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–6, 2018.

[12] Yanran Ding, Chuanzheng Li, and Hae-Won Park. Kino-dynamic motion planning for multi-legged robot jumping via mixed-integer convex program. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3998–4005, 2020.

[13] OSU DRL. cassie-mujoco-sim, 2023. URL https://github.com/osudrl/cassie-mujoco-sim.

[14] Luke Drnach and Ye Zhao. Robust trajectory optimization over uncertain terrain with stochastic complementarity. *IEEE Robotics and Automation Letters*, 6(2):1168–1175, 2021.

[15] Alejandro Escontrela, Xue Bin Peng, Wenhao Yu, Tingnan Zhang, Atil Iscen, Ken Goldberg, and Pieter Abbeel. Adversarial motion priors make good substitutes for complex reward functions. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 25–32, 2022.

[16] Gilbert Feng, Hongbo Zhang, Zhongyu Li, Xue Bin Peng, Bhuvan Basireddy, Linzhu Yue, Zhitao Song, Lizhi Yang, Yunhui Liu, Koushil Sreenath, et al. Genloco: Generalized locomotion controllers for quadrupedal robots. *arXiv preprint arXiv:2209.05309*, 2022.

[17] Zipeng Fu, Ashish Kumar, Jitendra Malik, and Deepak Pathak. Minimizing energy consumption leads to the emergence of gaits in legged robots. *arXiv preprint arXiv:2111.01674*, 2021.

[18] Zipeng Fu, Xuxin Cheng, and Deepak Pathak. Deep whole-body control: learning a unified policy for manipulation and locomotion. *arXiv preprint arXiv:2210.10044*, 2022.

[19] Scott Gilroy, Derek Lau, Lizhi Yang, Ed Izaguirre, Kristen Biermayer, Anxing Xiao, Mengti Sun, Ayush Agrawal, Jun Zeng, Zhongyu Li, et al. Autonomous navigation for quadrupedal robots with optimized jumping through constrained obstacles. In *2021 IEEE 17th International Conference on Automation Science and Engineering (CASE)*, pages 2132–2139, 2021.

[20] Dip Goswami and Prahlad Vadakkepat. Planar bipedal jumping gaits with stable landing. *IEEE Transactions on Robotics*, 25(5):1030–1046, 2009.

[21] Tuomas Haarnoja, Sehoon Ha, Aurick Zhou, Jie Tan, George Tucker, and Sergey Levine. Learning to walk via deep reinforcement learning. *arXiv preprint arXiv:1812.11103*, 2018.

[22] Ross Hartley, Maani Ghaffari, Ryan M Eustice, and Jessy W Grizzle. Contact-aided invariant extended kalman filtering for robot state estimation. *The International Journal of Robotics Research*, 39(4):402–430, 2020.

[23] Masato Hirose and Kenichi Ogawa. Honda humanoid robots development. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 365(1850):11–19, 2007.

[24] Xiaoyu Huang, Zhongyu Li, Yanzhen Xiang, Yiming Ni, Yufeng Chi, Yunhao Li, Lizhi Yang, Xue Bin Peng, and Koushil Sreenath. Creating a dynamic quadrupedal robotic goalkeeper with reinforcement learning. *arXiv preprint arXiv:2210.04435*, 2022.

[25] Se Hwan Jeon, Sangbae Kim, and Donghyun Kim. Online optimal landing control of the mit mini cheetah. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 178–184, 2022.

[26] Gwanghyeon Ji, Juhyeok Mun, Hyeongjun Kim, and Jemin Hwangbo. Concurrent training of a control policy

[2]Ananye Agarwal, Ashish Kumar, Jitendra Malik, 和 Deepak Pathak。利用自我中心视觉在挑战性地形中进行腿部运动。*arXiv*预印本 *arXiv:2211.07638*, 2022年。[3] Ryan Batke, Fangzhou Yu, Jeremy Dao, Jonathan Hurst, Ross L Hatton, Alan Fern, 和 Kevin Green。针对强化学习优化单刚体模型的双足机动。载于*2022*年*IEEE-RAS*第*21*届国际仿人机器人会议（*Humanoids*），第714–721页，2022年。[4] Guillaume Bellegarda 和 Auke Ijspeert。CPG-RL: 学习四足运动的中枢模式发生器。*IEEE*机器人与自动化快报，7(4):12547–12554，2022年。[5] Guillaume Bellegarda, Yiyu Chen, Zhuochen Liu, 和 Quan Nguyen。通过深度强化学习实现四足机器人的鲁棒高速奔跑。载于*2022*年*IEEE/RSJ*智能机器人与系统国际会议（*IROS*），第10364–10370页，2022年。[6]Miroslav Bogdanovic, Majid Khadiv, 和 Ludovic Righetti。利用轨迹优化演示实现鲁棒运动的无模型强化学习。机器人与人工智能前沿，9，2022年。[7]Guillermo A Castillo, Bowen Weng, Wei Zhang, 和 Ayonga Hereid。在3D Digit双足机器人上使用强化学习的鲁棒反馈运动策略设计。载于*2021*年*IEEE/RSJ*智能机器人与系统国际会议（*IROS*），第5136–5143页，2021年。[8] Iordanis Chatzinikolaidis, Yangwei You, 和 Zhibin Li。使用可解析接触模型的可变地面运动接触隐式轨迹优化。*IEEE*机器人与自动化快报，5(4):6357–6364，2020年。[9] Hua Chen, Bingheng Wang, Zejun Hong, Cong Shen, Patrick M Wensing, 和 Wei Zhang。轮式双足机器人跳跃的欠驱动运动规划与控制。*IEEE*机器人与自动化快报，6(2):747–754，2020年。[10] Hongkai Dai, Andr´es Valenzuela, 和 Russ Tedrake。基于质心动力学与完整运动学的全身运动规划。载于*2014*年*IEEE-RAS*国际仿人机器人会议，第295–302页，2014年。[11] Yanran Ding, Chuanzheng Li, 和 Hae-Won Park。基于混合整数凸优化的单腿动态运动规划。载于*2018*年*IEEE/RSJ*智能机器人与系统国际会议（*IROS*），第1–6页，2018年。[12] Yanran Ding, Chuanzheng Li, 和 Hae-Won Park。基于混合整数凸规划的多足机器人跳跃运动学动态运动规划。载于*2020*年*IEEE/RSJ*智能机器人与系统国际会议（*IROS*），第3998–4005页，2020年。[13] OSU DRL。cassie-mujoco-sim，2023年。网址：https://github.com/osudrl/cassie-mujoco-sim。[14] Luke Drnach 和 Ye Zhao。鲁棒轨迹优化

tion over uncertain terrain with stochastic complementar- ity. *IEEE*机器人与自动化快报，6(2):1168– 1175, 2021.[15]Alejandro Escontrela, Xue Bin Peng, Wenhao Yu, Tingnan Zhang, Atil Iscen, Ken Goldberg, and Pieter Abbeel. 对抗性运动先验是复杂奖励函数的良好替代品。 In *2022*年*IEEE/RSJ*国际智能机器人与系统大会（*IROS*）, pages 25–32, 2022.[16] Gilbert Feng, Hongbo Zhang, Zhongyu Li, Xue Bin Peng, Bhuvan Basireddy, Linzhu Yue, Zhitao Song, 杨立志, Yunhui Liu, Koushil Sreenath, et al. Genloco: 四足机器人通用运动控制器。*arXiv*预印本 *arXiv:2209.05309*, 2022.[17] Zipeng Fu, Ashish Kumar, Jitendra Malik, and Deepak Pathak. 最小化能耗导致腿式机器人步态的出现。 *arXiv*预印本 *arXiv:2111.01674*, 2021.[18]Zipeng Fu, Xuxin Cheng, and Deepak Pathak. 深度全身控制：学习用于操作和运动的统一策略。 *arXiv*预印本 *arXiv:2210.10044*, 2022.[19]Scott Gilroy, Derek Lau, 杨立志, Ed Izaguirre, Kristen Biermayer, Anxing Xiao, Mengti Sun, Ayush Agrawal, Jun Zeng, Zhongyu Li, et al. 四足机器人通过受限障碍物的优化跳跃自主导航。 In *2021*年*IEEE*第*17*届自动化科学与工程国际会议（*CASE*）, pages 2132–2139, 2021.[20] Dip Goswami and Prahlad Vadakkepat. 具有稳定着陆的平面双足跳跃步态。 *IEEE*机器人学汇刊, 25(5):1030–1046, 2009.[21]Tuomas Haarnoja, Sehoon Ha, Aurick Zhou, Jie Tan, George Tucker, and Sergey Levine. 通过深度强化学习学会行走。 *arXiv*预印本 *arXiv:1812.11103*, 2018.[22]Ross Hartley, Maani Ghaffari, Ryan M Eustice, and Jessy W Grizzle. 用于机器人状态估计的接触辅助不变扩展卡尔曼滤波。 国际机器人研究杂志, 39(4):402–430, 2020.[23] Masato Hirose and Kenichi Ogawa. 本田仿人机器人开发。 皇家学会哲学汇刊*A*辑：数学、物理与工程科学, 365(1850):11–19, 2007.[24]黄晓宇, Zhongyu Li, Yanzhen Xiang, 倪一鸣, Yufeng Chi, Yunhao Li, 杨立志, Xue Bin Peng, and Koushil Sreenath. 利用强化学习创建动态四足机器人守门员。 *arXiv*预印本 *arXiv:2210.04435*, 2022.[25]Se Hwan Jeon, Sangbae Kim, and Donghyun Kim. MIT迷你猎豹的在线最优着陆控制。 In*2022*年国际机器人与自动化会议（*ICRA*）, pages 178–184, 2022.[26]Gwanghyeon Ji, Juhyeok Mun, Hyeongjun Kim, and Jemin Hwangbo. 控制策略的并行训练

and a state estimator for dynamic and robust legged locomotion. *IEEE Robotics and Automation Letters*, 7 (2):4630–4637, 2022.

[27] Yandong Ji, Zhongyu Li, Yinan Sun, Xue Bin Peng, Sergey Levine, Glen Berseth, and Koushil Sreenath. Hierarchical reinforcement learning for precise soccer shooting skills using a quadrupedal robot. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1479–1486, 2022.

[28] Dmitry Kalashnikov, Jacob Varley, Yevgen Chebotar, Benjamin Swanson, Rico Jonschkowski, Chelsea Finn, Sergey Levine, and Karol Hausman. Mt-opt: Continuous multi-task robotic reinforcement learning at scale. *arXiv preprint arXiv:2104.08212*, 2021.

[29] Benjamin Katz, Jared Di Carlo, and Sangbae Kim. Mini cheetah: A platform for pushing the limits of dynamic quadruped control. In *2019 international conference on robotics and automation (ICRA)*, pages 6295–6301, 2019.

[30] Daniel E Koditschek and Martin Buehler. Analysis of a simplified hopping robot. *The International Journal of Robotics Research*, 10(6):587–605, 1991.

[31] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. Rma: Rapid motor adaptation for legged robots. *arXiv preprint arXiv:2107.04034*, 2021.

[32] Ashish Kumar, Zhongyu Li, Jun Zeng, Deepak Pathak, Koushil Sreenath, and Jitendra Malik. Adapting rapid motor adaptation for bipedal robots. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1161–1168, 2022.

[33] Benoit Landry, Joseph Lorenzetti, Zachary Manchester, and Marco Pavone. Bilevel optimization for planning through contact: A semidirect method. In *Robotics Research: The 19th International Symposium ISRR*, pages 789–804, 2022.

[34] Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5 (47):eabc5986, 2020.

[35] Quanyi Li, Zhenghao Peng, Haibin Wu, Lan Feng, and Bolei Zhou. Human-ai shared control via policy dissection. *arXiv preprint arXiv:2206.00152*, 2022.

[36] Zhongyu Li, Christine Cummings, and Koushil Sreenath. Animated cassie: A dynamic relatable robotic character. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3739–3746, 2020.

[37] Zhongyu Li, Xuxin Cheng, Xue Bin Peng, Pieter Abbeel, Sergey Levine, Glen Berseth, and Koushil Sreenath. Reinforcement learning for robust parameterized locomotion control of bipedal robots. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2811–2817, 2021.

[38] Tzu-Yuan Lin, Ray Zhang, Justin Yu, and Maani Ghaffari. Legged robot state estimation using invariant kalman filtering and learned contact events. In *5th Annual Conference on Robot Learning*, 2021.

[39] Zachary Manchester and Scott Kuindersma. Variational contact-implicit trajectory optimization. In *Robotics Research: The 18th International Symposium ISRR*, pages 985–1000, 2020.

[40] Gabriel Margolis, Ge Yang, Kartik Paigwar, Tao Chen, and Pulkit Agrawal. Rapid locomotion via reinforcement learning. In *Robotics: Science and Systems*, 2022.

[41] Gabriel B Margolis and Pulkit Agrawal. Walk these ways: Tuning robot control for generalization with multiplicity of behavior. *Conference on Robot Learning*, 2022.

[42] Gabriel B Margolis, Tao Chen, Kartik Paigwar, Xiang Fu, Donghyun Kim, Sang bae Kim, and Pulkit Agrawal. Learning to jump from pixels. In *5th Annual Conference on Robot Learning*, 2021.

[43] Takahiro Miki, Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62):eabk2822, 2022.

[44] Mark P Moresi, Elizabeth J Bradshaw, David Greene, and Geraldine Naughton. The assessment of adolescent female athletes using standing and reactive long jumps. *Sports Biomechanics*, 10(02):73–84, 2011.

[45] Chuong Nguyen, Lingfan Bao, and Quan Nguyen. Continuous jumping for legged robots on stepping stones via trajectory optimization and model predictive control. *arXiv preprint arXiv:2204.01147*, 2022.

[46] Quan Nguyen, Matthew J Powell, Benjamin Katz, Jared Di Carlo, and Sangbae Kim. Optimized jumping on the mit cheetah 3 robot. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 7448–7454, 2019.

[47] Hae-Won Park, Patrick M Wensing, Sangbae Kim, et al. Online planning for autonomous running jumps over obstacles in high-speed quadrupeds. *Robotics: Science and System*, 2015.

[48] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 3803–3810, 2018.

[49] Xue Bin Peng, Erwin Coumans, Tingnan Zhang, Tsang-Wei Lee, Jie Tan, and Sergey Levine. Learning agile robotic locomotion skills by imitating animals. *arXiv preprint arXiv:2004.00784*, 2020.

[50] Xue Bin Peng, Ze Ma, Pieter Abbeel, Sergey Levine, and Angjoo Kanazawa. Amp: Adversarial motion priors for stylized physics-based character control. *ACM Transactions on Graphics (TOG)*, 40(4):1–20, 2021.

[51] Samuel Pfrommer, Mathew Halm, and Michael Posa. Contactnets: Learning discontinuous contact dynamics with smooth, implicit representations. In *Conference on Robot Learning*, pages 2279–2291. PMLR, 2021.

[52] Michael Posa, Cecilia Cantu, and Russ Tedrake. A direct method for trajectory optimization of rigid bodies through contact. *The International Journal of Robotics*

和用于动态鲁棒腿式运动的状态估计器。 *IEEE*机器人与自动化快报, 7 (2):4630–4637, 2022.[27]Yandong Ji, Zhongyu Li, Yinan Sun, Xue Bin Peng, Sergey Levine, Glen Berseth, and Koushil Sreenath. 使用四足机器人进行精确足球射门技能的分层强化学习。 In *2022*年*IEEE/RSJ*国际智能机器人与系统大会（*IROS*）, pages 1479–1486, 2022.[28]Dmitry Kalashnikov, Jacob Varley, Yevgen Chebotar, Benjamin Swanson, Rico Jonschkowski, Chelsea Finn, Sergey Levine, and Karol Hausman. MT-Opt：大规模连续多任务机器人强化学习。 *arXiv*预印本 *arXiv:2104.08212*, 2021.[29] Benjamin Katz, Jared Di Carlo, and Sangbae Kim. Mini Cheetah：推动动态四足控制极限的平台。 In *2019*年国际机器人与自动化会议（*ICRA*）, pages 6295–6301, 2019.[30] Daniel E Koditschek and Martin Buehler. 简化跳跃机器人分析。国际机器人研究杂志 , 10(6):587–605, 1991.[31] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA：腿式机器人的快速运动适应。 *arXiv*预印本 *arXiv:2107.04034*, 2021.[32] Ashish Kumar, Zhongyu Li, Jun Zeng, Deepak Pathak, Koushil Sreenath, and Jitendra Malik. 为双足机器人调整快速运动适应。 In *2022*年*IEEE/RSJ*国际智能机器人与系统大会（*IROS*）, pages 1161–1168, 2022.[33] Benoit Landry, Joseph Lorenzetti, Zachary Manchester, and Marco Pavone. 用于接触规划的双层优化：一种半直接法。 In 机器人研究：第*19*届国际研讨会*ISRR*, pages 789–804, 2022.[34]Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. 在挑战性地形上学习四足运动。 科学机器人, 5 (47):eabc5986, 2020.[35] Quanyi Li, Zhenghao Peng, Haibin Wu, Lan Feng, and Bolei Zhou. 通过策略剖析实现人机共享控制。 *arXiv*预印本 *arXiv:2206.00152*, 2022.[36] Zhongyu Li, Christine Cummings, and Koushil Sreenath. Animated Cassie：一个动态可关联机器人角色。 In *2020*年*IEEE/RSJ*国际智能机器人与系统大会（*IROS*）, pages 3739–3746, 2020.[37] Zhongyu Li, Xuxin Cheng, Xue Bin Peng, Pieter Abbeel, Sergey Levine, Glen Berseth, and Koushil Sreenath. 用于双足机器人鲁棒参数化运动控制的强化学习。 In *2021*年*IEEE*国际机器人与自动化会议（*ICRA*）, pages 2811–2817, 2021.[38] Tzu-Yuan Lin, Ray Zhang, Justin Yu, and Maani Ghaffari. 使用不变卡尔曼滤波和学习接触事件的腿式机器人状态估计。 In 第五届年度机器人学习会议，2021年。[39] Zachary Manchester 和 Scott Kuindersma。变分接触隐式轨迹优化。载于 机器人研究：第*18*届国际研讨会*ISRR*，第985–1000页，2020年。[40] Gabriel Margolis、Ge Yang、Kartik Paigwar、Tao Chen 和 Pulkit Agrawal。基于强化学习的快速运动。载于机器人：科学与系统，2022年。[41]Gabriel B Margolis 和 Pulkit Agrawal。Walk these ways: Tuning robot control for generalization with multiplicity of behavior。载于 机器人学习会议，2022年。[42] Gabriel B Margolis、Tao Chen、Kartik Paigwar、Xiang Fu、Donghyun Kim、Sangbae Kim 和 Pulkit Agrawal。从像素学习跳跃。载于 第五届机器人学习年度会议，2021年。[43]Takahiro Miki、Joonho Lee、Jemin Hwangbo、Lorenz Wellhausen、Vladlen Koltun 和 Marco Hutter。学习四足机器人在野外鲁棒的感知运动。载于 科学机器人，7(62):eabk2822，2022年。[44] Mark P Moresi、Elizabeth J Bradshaw、David Greene 和 Geraldine Naughton。使用站立和反应性跳远评估青少年女性运动员。载于运动生物力学，10(02):73–84，2011年。[45] Chuong Nguyen、Lingfan Bao 和 Quan Nguyen。基于轨迹优化和模型预测控制的腿式机器人连续跳跃过踏脚石。载于*arXiv*预印本 *arXiv:2204.01147*，2022年。[46] Quan Nguyen、Matthew J Powell、Benjamin Katz、Jared Di Carlo 和 Sangbae Kim。MIT猎豹3号机器人的优化跳跃。载于 *2019*年国际机器人与自动化大会 *(ICRA)*，第7448–7454页，2019年。[47] Hae-Won Park、Patrick M Wensing、Sangbae Kim 等。高速四足机器人自主跑跳越障的在线规划。载于 机器人：科学与系统，2015年。[48] Xue Bin Peng、Marcin Andrychowicz、Wojciech Zaremba 和 Pieter Abbeel。带动力学随机化的机器人控制仿真到现实迁移。载于 *2018*年*IEEE*国际机器人与自动化大会 *(ICRA)*，第3803–3810页，2018年。[49] Xue Bin Peng、Erwin Coumans、Tingnan Zhang、Tsang-Wei Lee、Jie Tan 和 Sergey Levine。通过模仿动物学习敏捷的机器人运动技能。载于 *arXiv*预印本 *arXiv:2004.00784*，2020年。[50] Xue Bin Peng、Ze Ma、Pieter Abbeel、Sergey Levine 和 Angjoo Kanazawa。AMP: 用于风格化物理角色控制的对抗性运动先验。载于 *ACM*图形学交易 *(TOG)*，40(4):1–20，2021年。[51]Samuel Pfrommer、Mathew Halm 和 Michael Posa。ContactNets: 使用平滑隐式表示学习不连续接触动力学。载于 机器人学习会议，第2279–2291页。PMLR，2021年。[52]Michael Posa、Cecilia Cantu 和 Russ Tedrake。一种通过接触进行刚体轨迹优化的直接方法

Research, 33(1):69–81, 2014.

[53] M. H. Raibert, M. A. Chepponis, and H. Benjamin Brown. Experiments in balance with a 3d one-legged hopping machine. *International Journal of Robotics Research*, 3(2):75 – 92, June 1984.

[54] Diego Rodriguez and Sven Behnke. Deepwalk: Omnidirectional bipedal gait by deep reinforcement learning. In *2021 IEEE international conference on robotics and automation (ICRA)*, pages 3033–3039, 2021.

[55] Martin Rutschmann, Brian Satzinger, Marten Byl, and Katie Byl. Nonlinear model predictive control for rough-terrain robot hopping. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1859–1864, 2012.

[56] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[57] André Seyfarth, Hartmut Geyer, and Hugh Herr. Swing-leg retraction: a simple control model for stable running. *Journal of Experimental Biology*, 206(15):2547–2555, 2003.

[58] Yecheng Shao, Yongbin Jin, Xianwei Liu, Weiyan He, Hongtao Wang, and Wei Yang. Learning free gait transition for quadruped robots via phase-guided controller. *IEEE Robotics and Automation Letters*, 7(2):1230–1237, 2021.

[59] Jonah Siekmann, Srikar Valluri, Jeremy Dao, Lorenzo Bermillo, Helei Duan, Alan Fern, and Jonathan Hurst. Learning memory-based control for human-scale bipedal locomotion. *arXiv preprint arXiv:2006.02402*, 2020.

[60] Jonah Siekmann, Yesh Godse, Alan Fern, and Jonathan Hurst. Sim-to-real learning of all common bipedal gaits via periodic reward composition. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7309–7315, 2021.

[61] Laura Smith, J Chase Kew, Xue Bin Peng, Sehoon Ha, Jie Tan, and Sergey Levine. Legged robots that keep on learning: Fine-tuning locomotion policies in the real world. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 1593–1599, 2022.

[62] Laura Smith, Ilya Kostrikov, and Sergey Levine. A walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning. *arXiv preprint arXiv:2208.07860*, 2022.

[63] Zhitao Song, Linzhu Yue, Guangli Sun, Yihu Ling, Hongshuo Wei, Linhai Gui, and Yun-Hui Liu. An optimal motion planning framework for quadruped jumping. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11366–11373, 2022.

[64] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 5026–5033, 2012.

[65] Barkan Ugurlu, Jody A Saglia, Nikos G Tsagarakis, and Darwin G Caldwell. Hopping at the resonance frequency: A trajectory generation technique for bipedal robots with elastic joints. In *2012 IEEE International Conference on Robotics and Automation*, pages 1436–1443, 2012.

[66] Ivo Vatavuk and Zdenko Kovačić. Precise jump planning using centroidal dynamics based bilevel optimization. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3026–3032, 2021.

[67] Eric Vollenweider, Marko Bjelonic, Victor Klemm, Nikita Rudin, Joonho Lee, and Marco Hutter. Advanced skills through multiple adversarial motion priors in reinforcement learning. *arXiv preprint arXiv:2203.14912*, 2022.

[68] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Ken Goldberg, and Pieter Abbeel. Daydreamer: World models for physical robot learning. *arXiv preprint arXiv:2206.14176*, 2022.

[69] Zhaoming Xie, Glen Berseth, Patrick Clary, Jonathan Hurst, and Michiel van de Panne. Feedback control for cassie with deep reinforcement learning. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1241–1246, 2018.

[70] Zhaoming Xie, Patrick Clary, Jeremy Dao, Pedro Morais, Jonanthan Hurst, and Michiel Panne. Learning locomotion skills for cassie: Iterative design and sim-to-real. In *Conference on Robot Learning*, pages 317–329. PMLR, 2020.

[71] Xiaobin Xiong and Aaron D Ames. Bipedal hopping: Reduced-order model embedding via optimization-based control. In *International Conference on Intelligent Robots and Systems (IROS)*, pages 3821–3828, 2018.

[72] William Yang and Michael Posa. Impact invariant control with applications to bipedal locomotion. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5151–5158, 2021.

[73] Justin K Yim and Ronald S Fearing. Precision jumping limits from flight-phase control in salto-1p. In *2018 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 2229–2236, 2018.

[74] Fangzhou Yu, Ryan Batke, Jeremy Dao, Jonathan Hurst, Kevin Green, and Alan Fern. Dynamic bipedal maneuvers through sim-to-real reinforcement learning. *arXiv preprint arXiv:2207.07835*, 2022.

[75] Wenhao Yu, Visak CV Kumar, Greg Turk, and C Karen Liu. Sim-to-real transfer for biped locomotion. In *2019 ieee/rsj international conference on intelligent robots and systems (iros)*, pages 3503–3510, 2019.

[76] Chi Zhang, Wei Zou, Liping Ma, and Zhiqing Wang. Biologically inspired jumping robots: A comprehensive review. *Robotics and Autonomous Systems*, 124:103362, 2020.

[77] Yifan Zhu, Zherong Pan, and Kris Hauser. Contact-implicit trajectory optimization with learned deformable contacts using bilevel optimization. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9921–9927, 2021.

研究，33(1):69–81，2014。[53]M. H. Raibert、M. A. Chepponis 和 H. Benjamin Brown。三维单腿跳跃机的平衡实验。国际机器人研究杂志，3(2):75–92，1984年6月。[54] Diego Rodriguez 和 Sven Behnke。Deepwalk：通过深度强化学习实现全方位双足步态。载于*2021*年*IEEE*国际机器人与自动化会议*(ICRA)*，第3033–3039页，2021年。[55] Martin Rutschmann、Brian Satzinger、Marten Byl 和 Katie Byl。用于崎岖地形机器人跳跃的非线性模型预测控制。载于*2012*年*IEEE/RSJ*智能机器人与系统国际会议，第1859–1864页，2012年。[56] John Schulman、Filip Wolski、Prafulla Dhariwal、Alec Radford 和 Oleg Klimov。近端策略优化算法。*arXiv*预印本 *arXiv:1707.06347*，2017年。[57] Andr´e Seyfarth、Hartmut Geyer 和 Hugh Herr。摆腿回缩：一种用于稳定奔跑的简单控制模型。实验生物学杂志，206(15):2547–2555，2003年。[58] 邵业成、金永斌、刘贤伟、何伟彦、王洪涛和杨伟。通过相位引导控制器学习四足机器人的自由步态转换。*IEEE*机器人与自动化快报，7(2):1230–1237，2021年。[59] Jonah Siekmann、Srikar Valluri、Jeremy Dao、Lorenzo Bermillo、段和磊、Alan Fern 和 Jonathan Hurst。学习基于记忆的控制以实现人尺度双足运动。*arXiv*预印本 *arXiv:2006.02402*，2020年。[60] Jonah Siekmann、Yesh Godse、Alan Fern 和 Jonathan Hurst。通过周期性奖励组合实现所有常见双足步态的仿真到现实学习。载于*2021*年*IEEE*国际机器人与自动化会议*(ICRA)*，第7309–7315页，2021年。[61] Laura Smith、J Chase Kew、Xue Bin Peng、Sehoon Ha、Jie Tan 和 Sergey Levine。持续学习的腿式机器人：在真实世界中微调运动策略。载于*2022*年国际机器人与自动化大会*(ICRA)*，第1593–1599页，2022年。[62]Laura Smith、Ilya Kostrikov 和 Sergey Levine。公园漫步：通过无模型强化学习在20分钟内学会行走。*arXiv*预印本 *arXiv:2208.07860*，2022年。[63] Zhitao Song、Linzhu Yue、孙广利、凌一虎、魏宏硕、桂林海和刘云辉。一种用于四足跳跃的最优运动规划框架。载于*2022*年*IEEE/RSJ*国际智能机器人与系统大会（*IROS*），第11366–11373页，2022年。[64] Emanuel Todorov、Tom Erez 和 Yuval Tassa。MuJoCo：用于基于模型控制的物理引擎。载于*2012*年*IEEE/RSJ*国际智能机器人与系统大会，第5026–5033页，2012年。[65] Barkan Ugurlu、Jody A Saglia、Nikos G Tsagarakis 和 Darwin G Caldwell。共振频率跳跃：一种用于双足机器人的轨迹生成技术

弹性关节。在*2012 IEEE*国际机器人与自动化会议，第1436–1443页，2012年。[66] Ivo Vatavuk 和 Zdenko Kova ˇci´c。基于质心动力学的双层优化精确跳跃规划。在*2021 IEEE*国际机器人与自动化会议*(ICRA)*，第3026–3032页，2021年。[67]Eric Vollenweider、Marko Bjelonic、Victor Klemm、Nikita Rudin、Joonho Lee 和 Marco Hutter。通过强化学习中的多重对抗性运动先验实现高级技能。*arXiv*预印本 *arXiv:2203.14912*，2022年。[68] Philipp Wu、Alejandro Escontrela、Danijar Hafner、Ken Goldberg 和 Pieter Abbeel。Daydreamer：物理机器人学习的世界模型。*arXiv*预印本 *arXiv:2206.14176*，2022年。[69]Zhaoming Xie、Glen Berseth、Patrick Clary、Jonathan Hurst 和 Michiel van de Panne。基于深度强化学习的Cassie反馈控制。在*2018 IEEE/RSJ*国际智能机器人与系统大会（*IROS*），第1241–1246页，2018年。[70] Zhaoming Xie、Patrick Clary、J eremy Dao、Pedro Morais、Jonanthan Hurst 和 Michiel Panne。为Cassie学习运动技能：迭代设计与仿真到现实迁移。在机器人学习会议，第317–329页。PMLR，2020年。[71] Xiaobin Xiong 和 Aaron D Ames。双足跳跃：基于优化控制的降阶模型嵌入。在国际智能机器人与系统大会（*IROS*），第3821–3828页，2018年。[72]William Yang 和 Michael Posa。冲击不变控制及其在双足运动中的应用。在*2021 IEEE/RSJ*国际智能机器人与系统大会（*IROS*），第5151–5158页，2021年。[73] Justin K Yim 和 Ronald S Fearing。Salto-1p飞行阶段控制的精确跳跃极限。在*2018 IEEE/RSJ*国际智能机器人与系统大会（*IROS*），第2229–2236页，2018年。[74] Fangzhou Yu、Ryan Batke、Jeremy Dao、Jonathan Hurst、Kevin Green 和 Alan Fern。通过仿真到现实强化学习实现动态双足机动。*arXiv*预印本 *arXiv:2207.07835*，2022年。[75] Wenhao Yu、Visak CV Kumar、Greg Turk 和 C Karen Liu。双足运动的仿真到现实迁移。在*2019 IEEE/RSJ*国际智能机器人与系统大会（*IROS*），第3503–3510页，2019年。[76] Chi Zhang、Wei Zou、Liping Ma 和 Zhiqing Wang。仿生跳跃机器人：全面综述。机器人与自主系统，124:103362，2020年。[77] Yifan Zhu、Zherong Pan 和 Kris Hauser。使用双层优化与学习到的可变形接触进行接触隐式轨迹优化。在*2021 IEEE*国际机器人与自动化会议*(ICRA)*，第9921–9927页，2021年。

APPENDIX

A. Training in Stage 1

1) Reward: The reward design in Stage 1 is presented in Table II. In the first stage to initiate the training for a single jumping goal, we incentivize the robot to imitate the jumping-in-place animation. Therefore, the tracking rewards for motor position and foot height have overwhelming weights over others in order to accomplish a jump and stand still afterward. We also include the task completion term, but since the task is fixed in this stage, *i.e.*, $\mathbf{c} = \mathbf{0}$, this term is more to encourage the robot to jump in place and stabilize its pelvis orientation. We also have a smoothing term as a small fraction of the reward at this stage and do not have the change of action reward to prevent the robot from adopting a stationary behavior at the early stage of training.

2) Episode Design: In the initial training stage, the episode length is designed to have 750 timesteps corresponding to 23 seconds. The agent is asked to jump at $t = 0$ and to stand untill the end. If we allow the robot to stand at the beginning of the episode, the robot may focus on learning the easy standing skill and fail to explore the jumping maneuver. Furthermore, note that a jumping phase usually is less than 2 second but we have a 23-second episode. This is because the robot may learn jumping well but overlook the standing skill if the episode is short, which may result in the robot adopting an undesirable maneuver, such as continue hopping after landing. Having such a long episode can give the robot more incentive to learn a robust and stable standing skill in order to have a better return over the episode. The early termination conditions in Stage 1 are different than the multi-goal training stage, except for the falling-over condition. In this stage, the foot height tracking error bound E_e is smaller (0.22 m) while the task completion error bound E_t is much larger ($[1.0, 45°]$). This is because we want to push the robot to jump by lifting its feet at the initial stage of training and completing the task is not a big concern at this stage.

B. Details of Dynamics Randomization

The details of the dynamics parameters and randomization range used in this paper are listed in Table III. Note that the range of the noise is relatively small (such as $0.1°$ in joint position measurement and $0.5°$ in joint velocity) because we found that the onboard sensors on Cassie are reliable and therefore we use a smaller bound to reduce the training complexity. For the robot that has larger sensor noises, a larger bound of the noise during training is recommended.

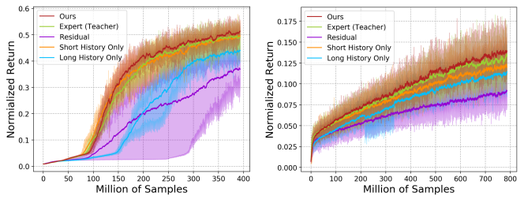
C. Details of Baseline Models

The details of the model structure we compared are listed as follow:

- **Ours** (Fig. 4a): the long-term I/O history is encoded with a CNN, while the short-term I/O history is provided directly as input to the base MLP. The policy direct outputs desired motion positions. The CNN encoder and the MLP base are jointly trained.
- **Residual** (Fig. 4b): the policy shares the same structure as the proposed one, but the policy output is a residual term added to the reference motor position at the current timestep, *i.e.*, $q_m^d = \mathbf{a}_t + q_m^r(t)$, which is used in [34, 70]. Please note that the policy has the reference motion as input.
- **Long History Only** (Fig. 4c): the policy only has a long-term I/O history encoded by a CNN, which is a baseline used in [31, 34]. Note that we still provide the robot feedback at the current timestep directly to the MLP base, as suggested by Peng et al. [48].
- **Short History Only** (Fig. 4d): the policy has short I/O history without the long-term I/O history CNN encoder, which is used in [37] and serves as a baseline in [32].
- **RMA/Teacher-Student**: an expert (teacher) policy (Fig. 4e) with access to privileged environment information (listed in Table III) is first trained using RL. The privileged information is encoded by an MLP into an 8D extrinsics vector. This expert policy is then used to *supervise* the training of an RMA (student) policy, which uses the base MLP copied from the expert policy, while using a long I/O history encoder to predict the teacher's extrinsic vector. This two-stage training scheme is used in [31, 34] and also adopted in other work such as [18, 26, 40].
- **A-RMA** (Fig. 4g): after the standard RMA training, the parameters of the long I/O history encoder are fixed, and the base MLP is further finetuned using RL as proposed by Kumar et al. [32]. Both RMA and A-RMA are also provided with a short I/O history which are newly added in this work for a fair comparison.

TABLE III: Dynamics Randomization Range

Parameters	Range
Floor Friction Ratio	[0.3, 3.0]
Joint Damping	[0.3, 4.0] Nms/rad
Spring Stiffness	[0.8, 1.2] × default
Link Mass	[0.5, 1.5] × default
Link Inertia	[0.7, 1.3] × default
Pelvis (Root) CoM Position	[-0.1, 0.1] m in $q_{x,y,z}$
Other Link CoM Position	[-0.05, 0.05] m + default
Motor PD Gains	[0.7, 1.3] × default
Motor Position Noise Mean	[-0.002, 0.002] rad
Motor Velocity Noise Mean	[-0.01, 0.01] rad/s
Gyro Rotation Noise Mean	[-0.002, 0.002] rad
Linear Velocity Estimation Error	[-0.04, 0.04] m/s
Communication Delay	[0, 0.025] sec



(a) Stage 1: Learning a Single Goal (b) Stage 2: Learning Multiple Goals

Fig. 10: Benchmark of learning curves trained by different policy structures trained with 3 random seeds in the early stages. The curves record the mean of normalized returns obtained using different seeds and the min and max among different seeds are the boundaries of the colored areas. The proposed method shows the best performance during the early stages of training including learning a single goal from scratch (Stage 1) and multiple goals in Stage 2.

D. Learning Performance in Early Stages

According to Fig. 5, at the training stages without dynamics randomization (Stage 1&2), our method shows similar, even a bit better, learning performance compared with the expert

附录

A. 阶段1训练

1) 奖励：阶段1的奖励设计如表II所示。在启动单跳目标训练的第一阶段，我们激励机器人模仿原地跳跃动画。因此，电机位置和脚部高度的跟踪奖励权重远高于其他项，以完成一次跳跃并随后保持静止。我们还包含了任务完成项，但由于此阶段任务固定，即，$\mathbf{c} = \mathbf{0}$，该项更多是鼓励机器人原地跳跃并稳定其骨盆朝向。在此阶段，我们还设置了一个平滑项，占奖励的一小部分，并且没有设置动作变化奖励，以防止机器人在训练早期采取静止行为。

2) 回合设计：在初始训练阶段，回合长度被设计为750个时间步，对应23秒。智能体被要求在 $t = 0$ 处跳跃，并站立至回合结束。如果允许机器人在回合开始时站立，机器人可能会专注于学习简单的站立技能，而未能探索跳跃动作。此外，请注意，跳跃阶段通常不到2秒，但我们有23秒的回合。这是因为如果回合较短，机器人可能能很好地学习跳跃，但会忽视站立技能，这可能导致机器人采取不良动作，例如着陆后继续跳跃。设置如此长的回合可以激励机器人学习稳健且稳定的站立技能，以便在回合中获得更好的回报。阶段1中的提前终止条件与多目标训练阶段不同，但摔倒条件除外。在此阶段，脚部高度跟踪误差界限 E_e 较小（0.22米），而任务完成误差界限 E_t 则大得多（$[1.0, 45°]$）。这是因为我们希望在训练初期通过抬脚来推动机器人跳跃，而在此阶段完成任务并非主要关注点。

B. 动力学随机化细节

本文所使用的动力学参数及随机化范围详情列于表III。请注意，噪声范围相对较小（例如关节位置测量中的 $0.1°$ 和关节速度中的 $0.5°$），这是因为我们发现Cassie机器人上的机载传感器可靠性较高，因此采用较小的边界以降低训练复杂度。对于传感器噪声较大的机器人，建议在训练过程中使用更大的噪声边界。

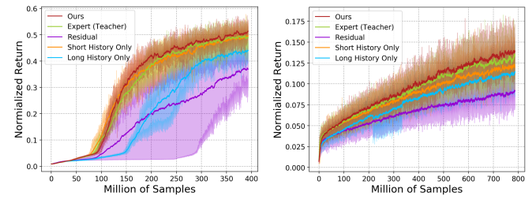
C. 基线模型细节

我们对比的模型结构详情如下：

- **我们的方法**（图4a）：长期输入输出历史通过卷积神经网络编码，而短期输入输出历史直接作为输入提供给基础多层感知机。策略直接输出期望运动位置。卷积神经网络编码器和MLP基础网络联合训练。
- **残差**（图4b）：策略结构与所提出的方法相同，但策略输出是一个残差项，该残差项被添加到

表III：动力学随机化范围

参数	范围
地面摩擦系数	[0.3, 3.0]
关节阻尼	[0.3, 4.0] Nms/rad
弹簧刚度	[0.8, 1.2] × 默认
连杆质量	[0.5, 1.5] × 默认
连杆惯性	[0.7, 1.3] ×默认
骨盆（根部）质心位置	[-0.1, 0.1] 米$q_{x,y,z}$
其他连杆质心位置	[-0.05, 0.05] 米 + 默认
电机PD增益	[0.7, 1.3] ×默认
电机位置噪声均值	[-0.002, 0.002] 弧度
电机速度噪声均值	[-0.01, 0.01] 弧度/秒
陀螺仪旋转噪声均值	[-0.002, 0.002]弧度
线速度估计误差	[-0.04, 0.04] 米/秒
通信延迟	[0, 0.025] 秒



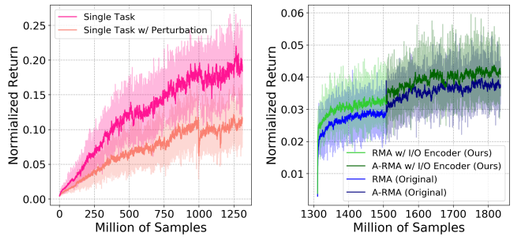
(a) 阶段1：学习单一目标 (b) 阶段2：学习多目标

图10：不同策略结构在早期阶段使用3个随机种子训练的学习曲线基准。曲线记录了使用不同种子获得的归一化回报均值，不同种子之间的最小值和最大值构成彩色区域的边界。所提出的方法在训练的早期阶段（包括从零开始学习单一目标的阶段1和阶段2中的多目标学习）均表现出最佳性能。

当前时间步的参考电机位置，即，$q_m^d = \mathbf{a}_t + q_m^r(t)$，该方法在 [34, 70]中使用。请注意，策略将参考运动作为输入。
- **仅长历史**（图4c）：策略仅具有由卷积神经网络编码的长期输入输出历史，这是 [31, 34]中使用的基线。注意，我们仍然按照Peng等人的建议 [48]，将当前时间步的机器人反馈直接提供给MLP基础网络。
- **仅短历史**（图4d）：策略具有短期输入输出历史，但没有长期输入输出历史卷积神经网络编码器，该方法在 [37]中使用，并作为 [32]中的基线。
- **RMA/教师-学生**：首先使用强化学习训练一个能够访问特权环境信息（列于表III）的专家（教师）策略（图4e）。特权信息由多层感知机编码为8维外部向量。然后，该专家策略用于监督RMA（学生）策略的训练，该学生策略使用从专家策略复制而来的MLP基础网络，同时使用长期输入输出历史编码器来预测教师外部向量。这种两阶段训练方案在 [31, 34] 中使用，并在其他工作中也被采用，例如 [18, 26, 40]。
- **A-RMA** （图4g）：在标准RMA训练之后，固定长期输入输出历史编码器的参数，并按照Kumar等人 [32]提出的方法，使用强化学习进一步微调MLP基础网络。RMA和A-RMA也都提供了短期输入输出历史，这是本工作中为公平比较而新增的。

D. 早期阶段的学习性能

根据图5，在没有动力学随机化的训练阶段（阶段1和2），我们的方法表现出与能够获取特权环境信息的专家策略相似甚至略优的学习性能。



(a) Learning Single Goal with Domain Randomization (b) Learning with Different Memory Encoders for RMAs

Fig. 11: Additional Learning Curves.

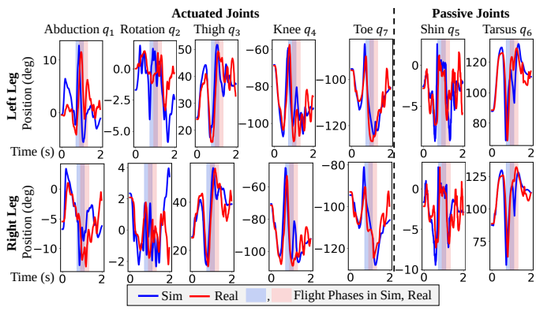


Fig. 12: The profiles of the robot's joint positions when it is commanded to jump to 0.44 m-tall elevation while forward 0.88 m in simulation and the real world, using the discrete-terrain policy.

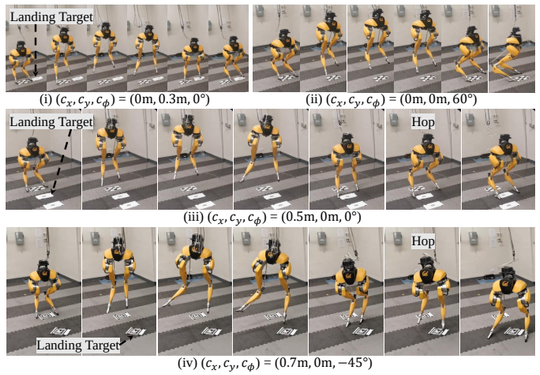


Fig. 13: Additional experiments show Cassie jumping to different targets with the single flat-ground policy.

(Stage 1, Fig. 10a), the learning performance using short history only and long history only (blue curve) show a similar learning performance in a more complex multi-goal training stage (Stage 2).

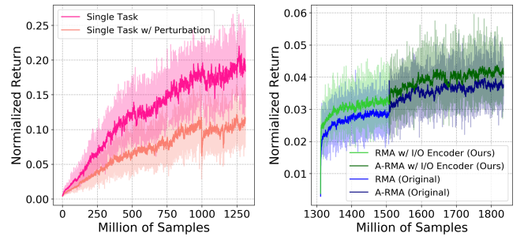
E. Additional Learning Curves

The learning curves for single-goal policies with dynamics randomization detailed in Sec. IV-E is recorded in Fig. 11a. Training RMAs with different long-history encoders are recorded in Fig. 11b. The RMA used in [31, 32] (Original) has a different structure of the long-term I/O encoder (1D CNN) than the one used in this work. It has 3 hidden layers and the [kernel size, filter size, stride size] of each layer is $[8, 32, 4]$, $[5, 32, 1]$, and $[5, 32, 1]$, with zero padding, respectively.

F. Additional Hardware Experiments

More experiment results are presented in Fig. 12 and Fig. 13. It shows the capacity of the flat-ground policy to accomplish more challenging jumping tasks on the real robot Cassie.

policy which has the access to the privileged environment information. This is because the long-term I/O history is able to provide more information than the dynamics parameters used in the expert policy, such as the robot's take-off trajectory which will be useful to determine a better landing maneuver. Although the policy with short history only (orange curve) shows a faster learning curve at the initial stage of training



(a) 学习单目标与动-主要随机化 (b) 面向不同记忆编码器的学习用于RMAs

图11：附加学习曲线。

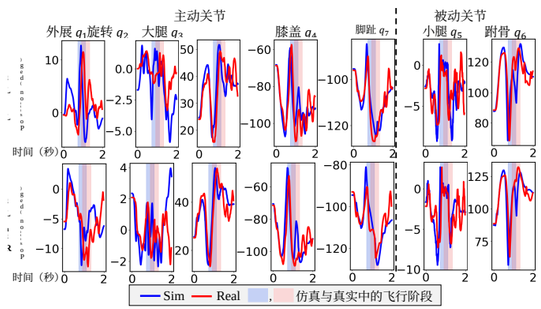


图12：使用离散地形策略，在仿真和真实世界中，当机器人被指令向前跳跃0.88米并跳至0.44米高的抬升目标时，其关节位置的轮廓图。

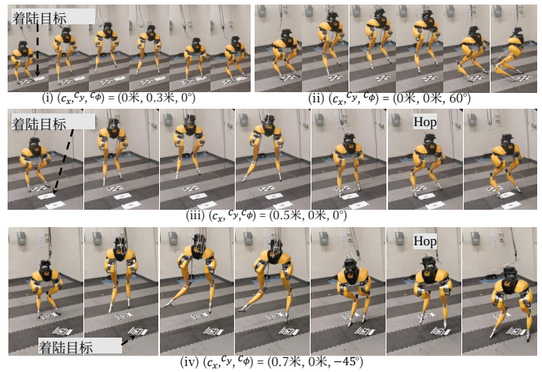


图13：附加实验展示了Cassie使用单一平地策略跳跃至不同目标的情况。

（阶段1，图10a），在更复杂的多目标训练阶段（阶段2）中，仅使用短历史和仅使用长历史（蓝色曲线）的学习性能表现出相似的学习性能。

E. 附加学习曲线

第IV-E节详述的带有动力学随机化的单目标策略的学习曲线记录在图11a中。使用不同长历史编码器训练RMAs的记录如图11b所示。 [31, 32] （原始）中使用的RMA的长期输入输出编码器（一维卷积神经网络）结构与本文使用的不同。它有3个隐藏层，每层的[卷积核大小、滤波器大小、步长] 分别为 $[8, 32, 4]$,$[5, 32, 1]$, 和 $[5, 32, 1]$, ， 并采用零填充。

F. 附加硬件实验

更多实验结果展示在图12和图13中。这显示了平地策略在真实机器人Cassie上完成更具挑战性的跳跃任务的能力。

这是因为长期输入输出历史能够提供比专家策略中使用的动力学参数更多的信息，例如机器人的起飞轨迹，这将有助于确定更优的着陆机动。尽管仅使用短历史的策略（橙色曲线）在训练初始阶段显示出更快的